



Edge on Kubernetes

Last updated: Tuesday, December 09, 2025

This content applies to release 2025 of Cumulocity.

Specifications contained herein are subject to change and these changes will be reported in subsequent versions.

Copyright © 2025 Cumulocity GmbH.

The name Cumulocity GmbH and all Cumulocity GmbH product names are either trademarks or registered trademarks of Cumulocity GmbH and/or its subsidiaries and/or its affiliates and/or their licensors. Other company and product names mentioned herein may be trademarks of their respective owners.

This software may include portions of third-party products. Third-party terms are set out in a 3rd-party-licenses file linked to or included with each installation package.

Table of Contents

Table of Contents	3
EDGE ON KUBERNETES	5
OVERVIEW	5
KUBERNETES OPERATOR	5
EDGE OPERATOR	6
VERSIONS	6
QUICKSTART	7
INSTALLING EDGE ON KUBERNETES	8
PREREQUISITES	8
SPECIAL INSTRUCTIONS FOR K3S	9
CONFIGURING PROXY	10
CONFIGURING STORAGE	11
PERSISTENT VOLUME CLAIMS MADE BY THE EDGE OPERATOR	12
STATIC PROVISIONING OF PVS	12
INSTALLING THE EDGE OPERATOR	13
INSTALLING THE EDGE OPERATOR (OFFLINE)	14
INSTALLING A PRIVATE REGISTRY	14
UPDATE /ETC/HOSTS TO RESOLVE THE DOMAIN	14
UPDATE COREDNS CONFIGURATION	14
TRUST THE PRIVATE REGISTRY'S CERTIFICATE	15
DOWNLOAD AND PUBLISH REQUIRED SOFTWARE TO THE PRIVATE REGISTRY	15
UPDATE CUSTOM-ENVIRONMENT-VARIABLES CONFIGMAP	16
INSTALLING THE EDGE OPERATOR	17
INSTALLING EDGE	17
VERIFYING THE EDGE INSTALLATION	17
ACCESSING EDGE	18
ASSIGNING AN EXTERNAL IP	19
ACCESSING EDGE USING THE DOMAIN NAME	19
HOW TO RESET OR CHANGE YOUR PASSWORD	22
HOW TO ACCESS PAGES USING URLS	22
ACCESSING LOGS	23
EDGE CUSTOM RESOURCE	24
CONFIGURATION	24
SPECIFICATION	24
TLS/SSL SECRET	27
CLOUD TENANT	28
CUMULOCITY CORE	29
MONGODB	30
RESOURCE LIMITS SPECIFICATION	31
MONGODB CREDENTIALS SECRET	32
EXTERNALLY HOSTED MONGODB TLS/SSL SECRET	33
MONGODB STORAGE SIZE	33
MICROSERVICES	33
CONNECTING EDGE TO THE CLOUD	35
REGISTERING EDGE IN THE CLOUD TENANT	35
UPGRADING EDGE REMOTELY	35
UPDATING EDGE APPLIANCES USING BULK OPERATIONS	38
DOWNLOADING DIAGNOSTICS REMOTELY	38
MANAGING SOFTWARE REMOTELY	39
INSTALLING SOFTWARE	39
UPDATING SOFTWARE	39
REMOVING SOFTWARE	40

DATA EXCHANGE USING DATA BROKER	40
MANAGING EDGE	42
MODIFYING EDGE	42
EXAMPLE CONFIGURATION CHANGE	42
UPGRADING EDGE	42
STARTING THE UPGRADE	42
UPGRADING EDGE REMOTELY	42
RESTARTING THE EDGE OPERATOR	42
UPGRADING THE EDGE OPERATOR	43
UNINSTALLING EDGE	43
UNINSTALLING THE EDGE OPERATOR	43
EDGE OPERATIONS	45
MONITORING	45
CUMULOCITY CORE METRICS	45
APAMA METRICS	45
MONGODB METRICS	47
STATEFULSETS METRICS	49
DEPLOYMENTS METRICS	49
POD METRICS (CPU AND MEMORY USAGE)	49
INSTALLING AND CONFIGURING MONITORING TOOLS	50
INSTALLING PROMETHEUS	50
INSTALLING GRAFANA	50
MONITORING THE EDGE METRICS FROM YOUR CLOUD TENANT	51
TROUBLESHOOTING	53
EDGE CR STATUS VALIDATIONLOOPBACKOFF	53
EDGE CR STATUS INSTALLLOOPBACKOFF OR UPDATELOOPBACKOFF	53
PENDING EXTERNAL CUMULOCITY-CORE SERVICE IP	54
EDGE NOT SHOWING UP AS A DEVICE IN THE CLOUD TENANT	54
DEPLOYMENT ARCHITECTURE	56
CONFIGURING EDGE	57
CONFIGURING REMOTE CONNECTIVITY	57
CONFIGURING THE EMAIL SERVER	57
CONNECTING DEVICES TO EDGE	58
INTRODUCTION	58
USING EDGE	59
GETTING STARTED	59
BRANDING	59
DEVELOPING CUMULOCITY WEB APPLICATIONS	59
WORKING WITH DATAHUB	60
ANALYTICAL QUERYING WITH DATAHUB EDGE	60
EDGE ON KUBERNETES	61
OVERVIEW	61
KUBERNETES OPERATOR	61
EDGE OPERATOR	62
VERSIONS	62

EDGE ON KUBERNETES

This document describes the conceptual aspects of [Cumulocity Edge](#) on Kubernetes, the single-server variant of the Cumulocity platform. For more information about Edge, see [Cumulocity Edge](#).

OVERVIEW

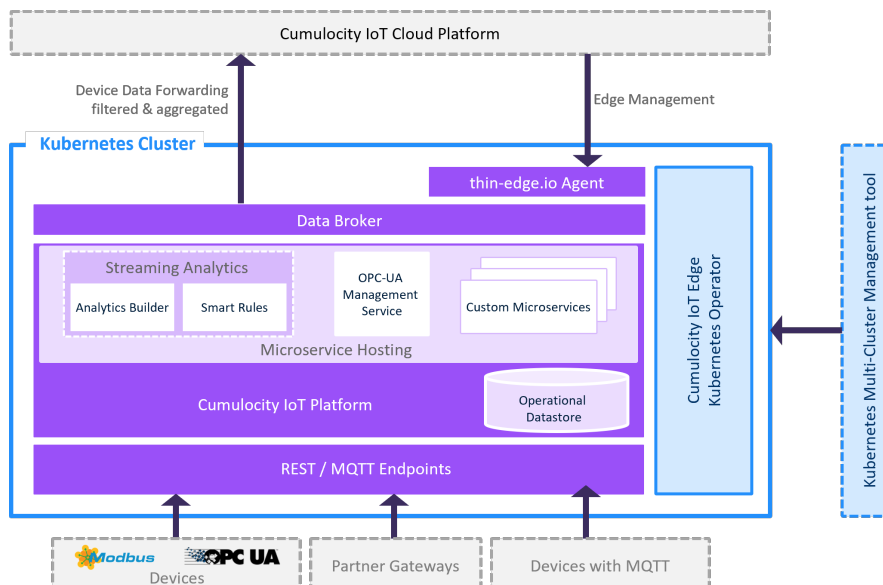
Edge on Kubernetes represents a cloud-native solution for the delivery, deployment, and management of Edge. It retains the functionalities of the Edge appliance VM, encompassing similar benefit like autonomy, data reduction, and reactivity.

Kubernetes offers an efficient platform for deploying, scaling, and managing containerized applications using a centralized control plane. Given this, containers orchestrated by Kubernetes have become standard in contemporary IT, fitting a broad spectrum of deployment contexts. Edge on Kubernetes capitalizes on these orchestration and management benefits, streamlining the operations of the Edge servers with other containerized applications in your deployment landscape.

Edge on Kubernetes captures all the highlighted advantages of a cloud-native deployment strategy by using a Kubernetes Operator, known as the Edge operator. This Edge operator serves as the central controller and facilitates the deployment and management of the Edge on a single-node Kubernetes cluster. This equips you with:

- **Unified Management:** Manage Edge through an Edge Custom Resource (CR) in accordance with standard Kubernetes resources. This approach provides a unified and familiar interface for administrators and operators.
- **Desired State Management:** Specify the desired state of Edge through Edge CR. The Edge operator then ensures that the actual state matches the desired state, simplifying operations and reducing the need for manual intervention.
- **GitOps Friendly:** The declarative nature of CR aligns seamlessly with GitOps workflows. The entire Edge deployment state, including configurations, can be stored and versioned in Git, promoting transparency and traceability.
- **Automation:** Edge CR empowers you to automate various operational aspects of Edge like scaling, upgrades, and configuration.
- **Distribution:** Distribute and replicate the Edge deployments using Edge CR. Simplify the deployment and accelerate the adoption of Cumulocity platform across diverse environments.

Although the features remain on par with the Edge appliance VM, the deployment strategy undergoes a change, as illustrated in the diagram below:



KUBERNETES OPERATOR

Kubernetes Operators are a powerful concept and pattern for managing complex applications and services within Kubernetes clusters. The operators are software extensions to Kubernetes that make use of [custom resources](#) to manage applications and their components. The operators follow Kubernetes principles, notably the [control loop](#).

The integral part of the operator pattern is Custom Resource Definitions (CRDs), which allow you to define application-specific resources and its behavior. An operator uses these CRDs to understand how to manage and interact with the application.

The operators are particularly valuable for managing stateful applications, like Cumulocity. The operators automate best practices, routine tasks, and provide a consistent way to manage complex applications across different environments.

EDGE OPERATOR

The Edge operator automates the deployment and management of Edge on Kubernetes. The Edge operator manages a Custom Resource Definition (CRD) to extend the Kubernetes API for Edge. You can deploy and manage Edge on a Kubernetes cluster through Edge Custom Resource (CR). Modify the Edge CR file and use `kubectl apply -f` to apply the changes. The Edge operator receives these changes and installs or updates Edge accordingly.

The Edge operator enables you to install, configure, upgrade or downgrade, scale up or down Edge on Kubernetes. The Edge operator supports:

- Deploy Edge
- Version upgrade and downgrade
- Scale up and down resources
- Configure persistent storage
- Validate configurations

VERSIONS

Cumulocity Edge 10.18 uses the following versions:

Edge component	Version
Cumulocity platform	10.18.0
Streaming Analytics	10.18

QUICKSTART

This section helps you to quickly install Edge on a [Lightweight Kubernetes \(K3s\)](#) cluster with default options. For detailed instructions, see [Installing Edge on Kubernetes](#).

1. Verify that your hardware meets the requirements specified in [Prerequisites](#).
2. Run the command below to install K3s.

```
USER_NAME=$(whoami)
USER_HOME=$(eval echo ~${USER_NAME})
sudo sh -c '
touch /etc/sysctl.d/90-kubelet.conf && \
sed -i "/^vm\.\panic_on_oom=/d; /^vm\.\overcommit_memory=/d; /^kernel\.\panic=/d; /^kernel\.\panic_on_oops=/d" /etc/sysctl.d/90-kubelet.conf && \
printf "vm.panic_on_oom=0\nvm.overcommit_memory=1\nkernel.panic=10\nkernel.panic_on_oops=1\n" | tee -a /etc/sysctl.d/90-kubelet.conf && \

sysctl -p /etc/sysctl.d/90-kubelet.conf && \

curl -sL https://get.k3s.io | INSTALL_K3S_VERSION=v1.25.13+k3s1 sh -s - \
--write-kubeconfig-mode 644 \
--disable=traefik \
--protect-kernel-defaults true \
--kube-apiserver-arg=admission-control=ValidatingAdmissionWebhook,MutatingAdmissionWebhook && \

mkdir -p "${USER_HOME}/.kube && \
cp /etc/rancher/k3s/k3s.yaml "${USER_HOME}/.kube/config && \
chown "${USER_NAME}:${USER_HOME}" "${USER_HOME}/.kube/config && \
chmod 600 "${USER_HOME}/.kube/config && \

printf "\e[32mSuccessfully installed k3s!\e[0m\n" && \

k3s crictl pull rancher/klipper-lb:v0.4.4 && \
k3s crictl pull rancher/mirrored-metrics-server:v0.6.3 && \
k3s crictl pull rancher/local-path-provisioner:v0.0.24
'
```

3. Run the command below to install Helm v3.

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
```

4. Run the command below to install the Edge operator and provide the registry credentials when prompted.

```
curl -sL http://localhost:1313/docs/files/edge-k8s/c8yedge-operator-install.sh -O && bash ./c8yedge-operator-install.sh
```

5. Run the command below to apply Edge CR ([c8yedge-sample.yaml](#)) for installing Edge version **1018.0.1** named **c8yedge** with the domain **myown.iot.com**.

```
kubectl apply -f http://localhost:1313/docs/files/edge-k8s/c8yedge-sample.yaml
```

6. See [Verifying the Edge installation](#) and [Accessing Edge](#) to sign into Edge.

INSTALLING EDGE ON KUBERNETES

This documentation provides information about installing Cumulocity Edge using the Edge operator and accessing Cumulocity Edge. Use the [c8yedge.yaml](#) file which includes Edge CR and the secrets necessary to deploy Cumulocity Edge.

PREREQUISITES

Item	Details
Hardware	CPU: 6 cores RAM: 10 GB CPU Architecture: x86-64 Info: These are the minimum system requirements for deploying Edge. If a custom microservice requires additional resources, you must configure the system accordingly in addition to the minimum requirements. For example, if a custom microservice requires 2 CPU cores and 4 GB of RAM, then the Kubernetes node must have 8 CPU cores (6 cores for standard workloads + 2 cores for your microservice) and 14 GB of RAM (10 GB for standard workloads + 4 GB for your microservice). Important: MongoDB requires a CPU that supports AVX instructions. Ensure that the CPU type of the Kubernetes node supports AVX instructions. Use the command <code>sudo lscpu</code> to check whether the CPU supports AVX instructions.
Kubernetes	Version 1.25.x has been tested (with potential compatibility for subsequent versions) and is supported across the following platforms: - Lightweight Kubernetes (K3s). To enable the proper functioning of the Edge operator on K3s, you must install K3s with the following configuration options. For more information, see Special instructions for K3s. - Kubernetes (K8s) - Amazon Elastic Kubernetes Service (EKS) - Microsoft Azure Kubernetes Service (AKS) Info: Edge on Kubernetes has undergone testing on the Kubernetes platforms mentioned above, using the Containerd, CRI-O, and Docker container runtimes.
 IMPORTANT Edge on Kubernetes is tested and supported on single-node Kubernetes clusters.	
Helm version 3.x	Refer to Installing Helm for the installation instructions.
Disk space	Three static Persistent Volumes (PV) or a StorageClass configured with dynamic provisioning to bind. - 75 GB for the Persistent Volume Claim (PVC) made for MongoDB (configurable through the Custom Resource). - 10 GB for the Persistent Volume Claim (PVC) made for the Private Registry to host custom microservices. - 5 GB for the Persistent Volume Claim (PVC) made for application logs. For more information about configuring the storage, see Configuring storage.

Item	Details
Edge license file	To request the license file for Edge, contact product support In the email, you must include - Your company name, under which the license has been bought - The domain name (for example, myown.iot.com), where Edge will be reachable For more information, see Domain name validation for Edge license key generation.
The Edge operator registry credentials	You will receive the Edge operator registry credentials along with the Edge license.
TLS/SSL key and certificates	Optional. TLS/SSL private key and domain certificates in PEM format. Generate a TLS/SSL key pair and a Certificate Signing Request (CSR) according to your company policies, and submit it to your internal or external Certificate Authority (CA). When creating the CSR, in addition to providing the Common Name (CN), Organization (O), and other required details, you must specify the Subject Alternative Name (SAN) to request a multi-domain certificate. Ensure that the SAN includes the domain names for the "edge" tenant and Management tenant. If you plan to install Cumulocity DataHub, include its domain name as well. For instance, if your Edge domain is <i>myown.iot.com</i>, make sure <i>myown.iot.com</i>, <i>management.myown.iot.com</i>, and <i>datahub.myown.iot.com</i> are listed in the SAN field. Additionally, verify that the TLS/SSL certificate includes the complete certificate chain in the correct order.
Connect Edge to the cloud	Optional. To connect and manage one or more Edge deployments from your Cumulocity cloud tenant, you will need an active Cumulocity Standard tenant with a subscription plan that includes the <i>advanced-software-mgmt</i> microservice.

SPECIAL INSTRUCTIONS FOR K3S

To enable the proper functioning of the Edge operator on K3s, you must install K3s with the following configuration options.

Run the command below to install Kubernetes version 1.25.13:

```

USER_NAME=$(whoami)
USER_HOME=$(eval echo ~${USER_NAME})
sudo sh -c '
touch /etc/sysctl.d/90-kubelet.conf && \
sed -i "/^vm\.\panic_on_oom=/d; /^vm\.\overcommit_memory=/d; /^kernel\.\panic=/d; /^kernel\.\panic_on_oops=/d" /etc/sysctl.d/90-kubelet.conf && \
printf "vm\.\panic_on_oom=0\nvm\.\overcommit_memory=1\nkernel\.\panic=10\nkernel\.\panic_on_oops=1\n" | tee -a /etc/sysctl.d/90-kubelet.conf && \

sysctl -p /etc/sysctl.d/90-kubelet.conf && \

curl -sfL https://get.k3s.io | INSTALL_K3S_VERSION=v1.25.13+k3s1 sh -s - \
--write-kubeconfig-mode 644 \
--disable=traefik \
--protect-kernel-defaults true \
--kube-apiserver-arg=admission-control=ValidatingAdmissionWebhook,MutatingAdmissionWebhook && \

mkdir -p "${USER_HOME}/.kube && \
cp /etc/rancher/k3s/k3s.yaml "${USER_HOME}/.kube/config && \
chown "${USER_NAME}:" "${USER_HOME}/.kube/config && \
chmod 600 "${USER_HOME}/.kube/config && \

printf "\e[32mSuccessfully installed k3s!\e[0m\n" && \

k3s crictl pull rancher/klipper-lb:v0.4.4 && \
k3s crictl pull rancher/mirrored-metrics-server:v0.6.3 && \
k3s crictl pull rancher/local-path-provisioner:v0.0.24

```

For configuration options, see [K3s configuration options](#).

- Added `--disable=traefik` in the install command to disable Traefik to avoid port conflicts between Traefik and cumulocity-core service, as both are LoadBalancer type services which expose port 443.
- Added `--kube-apiserver-arg=admission-control=ValidatingAdmissionWebhook,MutatingAdmissionWebhook` to enable admission controllers. The flag is set to enable the `ValidatingAdmissionWebhook` and `MutatingAdmissionWebhook` admission controllers, as Edge requires them. See <https://kubernetes.io/docs/reference/access-authn-authz/admission-controllers/>.
- Added `--protect-kernel-defaults` true to protect the default kernel settings on the host system. It prevents modifications to critical kernel parameters by container workloads running in Kubernetes. For more information, see <https://docs.k3s.io/security/hardening-guide#host-level-requirements>.

INFO

To install a later version of Kubernetes, update the environment variable `INSTALL_K3S_VERSION`.

CONFIGURING PROXY

When Cumulocity Edge is deployed behind a proxy, it must be configured to communicate with external endpoints over the internet through the proxy server. To configure Edge to use a proxy, you must create or update a ConfigMap named `custom-environment-variables` in the `c8yedge` namespace (or the one you deployed Edge into) with the required proxy settings. The keys `http_proxy`, `https_proxy` and `socks_proxy` must be set to the URLs of the HTTP, HTTPS and Socks proxies, respectively. The key `no_proxy` must be set to specify a comma-separated list of domain suffixes, IP addresses, or CIDR ranges that Edge should bypass the proxy server for.

Here is an example of a ConfigMap with proxy settings:

```
##
## An optional ConfigMap to configure the Edge operator with
## - Proxy details when accessing external endpoints through a Proxy
## - TLS/SSL certificates to trust
##
## http_proxy, https_proxy and optionally socks_proxy must be configured with the relevant URLs.
## no_proxy must be configured with a comma-separated list of addresses or domains for which the proxy should be bypassed.
##
apiVersion: v1
kind: ConfigMap
metadata:
  ## The name is fixed and cannot be changed.
  name: custom-environment-variables
  ## Namespace name into which you installed the Edge operator.
  namespace: c8yedge
data:
  http_proxy: <HTTP Proxy URL>
  https_proxy: <HTTPS Proxy URL>
  socks_proxy: <SOCKS Proxy URL>

  ## A comma-separated list of addresses or domains for which the proxy will be bypassed.
  ## This must be configured with the specified entries, Edge domain name, Kubernetes Pod CIDR (Cluster Pod IP Address Range),
  ## Kubernetes Service CIDR (Cluster Service IP Address Range) and any other domains, hosts or IPs
  ## you want to bypass the proxy when accessed.
  no_proxy: 127.0.0.1,::1,localhost,svc.,cluster.local,cumulocity,<edge domain name, for example, myown.iot.com>,<kubernetes cluster IP range, for
  example, 10.43.0.0/16>

  ## TLS/SSL certificates in PEM format that the Edge operator can trust, in addition to those included in the default system trust store.
  ## You can provide multiple TLS/SSL certificates for trust by combining them into a single string.
  ca.crt: <CA-CERTIFICATES TO TRUST>
```

By configuring Edge with the appropriate proxy settings, you ensure that it can seamlessly communicate with external endpoints through the proxy server, allowing it to function effectively in environments where proxy usage is mandated.

The table below provides more information:

Field	Required	Type	Default	Description
http_proxy	No	String		Specifies the URL of the HTTP proxy to be used for network connections.
https_proxy	No	String		Specifies the URL of the HTTPS proxy to be used for secure network connections.
socks_proxy	No	String		Specifies the URL of a SOCKS proxy.
no_proxy	No	String		Specifies a comma-separated list of addresses or domains for which the proxy will be bypassed. This is configured with the specified entries, Edge domain name, Kubernetes Pod CIDR (Cluster Pod IP Address Range), Kubernetes Service CIDR (Cluster Service IP Address Range) and any other domains, hosts or IPs you want to bypass the proxy when accessed.
ca.crt	No	String		TLS/SSL certificates in PEM format that the Edge operator can trust, in addition to those included in the default system trust store. You can provide multiple TLS/SSL certificates for trust by combining them into a single string.

CONFIGURING STORAGE

Kubernetes makes physical storage devices available to your cluster in the form of two API resources, PersistentVolume and PersistentVolumeClaim.

A Persistent Volume (PV) is a storage resource in Kubernetes that is provisioned and managed independently from the Pods that use it. It provides a way to store data in a durable and persistent manner, even if the Pod that uses it is deleted or restarted.

PVs are typically used to store data that must be preserved across Pod restarts or rescheduling, such as databases or file systems. They can be backed by various storage technologies, such as local disks, network-attached storage (NAS), or cloud-based storage services.

To use a PV in Kubernetes, you must define a PersistentVolume object that describes the characteristics of the storage, such as capacity, access modes, and the storage-provider-specific details. Once the PV is created, you can create a PersistentVolumeClaim object that requests a specific amount of storage with specific access requirements. The Persistent Volume Claim (PVC) binds to a matching PV, and the Pod can then use the PVC to mount the storage and access the data.

By using PVs and PVCs, you can decouple the storage management from the application deployment, making it easier to manage and scale your applications in Kubernetes.

PVs represent cluster resources, while PVCs serve as requests for these resources and also serve as validation checks for the resource they request. Provisioning PVs can be done in two ways: statically or dynamically.

- **Static provisioning:** In this method, a cluster administrator manually creates PVs, specifying details about the actual storage available for cluster users. These PVs are registered in the Kubernetes API and are ready for consumption.
- **Dynamic provisioning:** When none of the statically created PVs match a PVC's requirements, the cluster can automatically provision storage on-demand, specifically tailored for the PVC. This dynamic provisioning relies on [StorageClasses](#). To trigger dynamic provisioning, the PVC must request a StorageClass, and the administrator must have set up and configured that class accordingly. Claims that request an empty string ("") for the class effectively disable dynamic provisioning for themselves. If no StorageClass is specified in a claim, it falls back to using a default StorageClass if one is configured in the cluster. To enable a default StorageClass, the cluster administrator must activate the `DefaultStorageClass` admission controller on the API server. This can be achieved, for instance, by ensuring that `DefaultStorageClass` is included in the comma-delimited, ordered list of values for the `--enable-admission-plugins` flag of the API server component. For more details on API server command-line flags,

refer to the [kube-apiserver](#) documentation.

PERSISTENT VOLUME CLAIMS MADE BY THE EDGE OPERATOR

The Edge operator requests three PVCs, as outlined in the table below. Each of these PVCs utilizes the StorageClass if specified within the `spec.storageClassName` field of the Edge CR.

- In case you omit the `spec.storageClassName`, the Edge operator requests PVCs without a StorageClass, thereby instructing Kubernetes to utilize the default StorageClass configured in the cluster.
- If you explicitly specify an empty StorageClass as `""`, the Edge operator requests PVCs with an empty StorageClass, thereby instructing Kubernetes to carry out static provisioning.
- Finally, if you specify the name of an existing StorageClass for which dynamic provisioning is enabled, the Operator requests PVCs with that class name, thereby instructing Kubernetes to utilize dynamic provisioning according to the specified class.

Persistent volume	Persistent Volume Claim	Description
75 GB	<code>mongod-data-edge-db-rs0-0</code>	Claimed by the MongoDB server to retain application data. The default size is 75 GB, but this value can be adjusted using the <code>spec.mongodb.resources.requests.storage</code> field in the Edge CR file.
10 GB	<code>microservices-registry-data</code>	Claimed by the private docker registry to store microservice images.
5 GB	<code>edge-logs</code>	Claimed by the Edge logging component to store the application and system logs.

To guarantee the retention of physical storage even after the PVC is deleted (for example, when Edge is deleted) and to enable future storage expansion if needed, it's crucial to configure the StorageClass and/or the PVs with the following settings:

1. **Reclaim Policy:** Ensure that the reclaim policy is set to `Retain`. This setting preserves the storage even after the PVC deletion.
2. **Volume Expansion:** Set the volume expansion option to `true`. This setting enables the storage to be expanded when necessary.

If these recommended settings are not configured in the StorageClass, in the Edge CR status you receive the warnings below:

- persistent volume reclaim policy of StorageClass [storage-class] is currently set to [Delete] instead of the recommended value [Retain]
- allow volume to expand setting of the StorageClass [storage-class] is currently set to [false] instead of the recommended value [true]

These warnings serve as reminders to adjust these settings for optimal storage management.

Kubernetes provides a variety of persistent volume types, but two specific types enable Pod containers to access either a Network File System (NFS) or the cluster node's local filesystem (often set up as a NFS drive mapped to a local folder). This configuration is especially prevalent in on-premises deployments.

STATIC PROVISIONING OF PVS

INFO

You can skip this section if your Kubernetes cluster is already configured for dynamic provisioning of PVs.

This section outlines the steps for configuring the Kubernetes cluster to enable Edge to utilize NFS as a source for the PVs. For additional storage options, refer to the [Kubernetes documentation](#).

- Storage provisioning by connecting directly to the NFS server via PV configuration
 - Download the [c8yedge-pv-nfs.yaml](#) file.

- Create and export the folders required for the 3 PVs defined in the `c8yedge-pv-nfs.yaml` file. Ensure that the user running Kubernetes server has read/write access to these folders.
- Run the command below:

```
kubectl apply -f c8yedge-pv-nfs.yaml
```

- Storage provisioning by mapping NFS drive to a local folder into the cluster node
 - Download the [c8yedge-pv-local-path.yaml](#) file.
 - Create the folders in the local file system or mount NFS folders required for the 3 PVs defined in the `c8yedge-pv-local-path.yaml` file. Ensure that the user running Kubernetes server has read/write access to these folders.
 - Run the command below:

```
kubectl apply -f c8yedge-pv-local-path.yaml
```

i INFO

Since you manually created the PVs, you must specify an empty StorageClass as `""` in the `spec.storageClassName` field of the Edge CR for Kubernetes to carry out static provisioning, thereby binding PVC claims made by the Edge operator.

INSTALLING THE EDGE OPERATOR

To begin, create a new single-node Kubernetes cluster with the Kubernetes distribution of your choice, and configure `kubectl` to use that cluster. See [Prerequisites](#) for the supported Kubernetes distributions and versions.

A script to install the Edge operator is available at [c8yedge-operator-install.sh](#).

To install the Edge operator, download and run the script, refer to a sample command below. Enter the version (`-v` option, for example, 1018.0.1) you want to install, registry hostname (`-r` option) and the registry credentials you received along with the license when prompted. Use `-h` option to display the usage details.

i INFO

If you are installing Edge from a local/private registry, provide the hostname (`-r` option) as `:` and the respective credentials when prompted.

```
curl -sL http://localhost:1313/docs/files/edge-k8s/c8yedge-operator-install.sh -O && bash ./c8yedge-operator-install.sh -v "1018.0.1" -r registry.c8y.io
```

Provide the Edge operator registry credentials in the prompt:

```
Enter username to access Edge operator registry:
Enter password to access Edge operator registry:
```

i INFO

To request the Edge registry credentials, [contact product support](#).

By default, the Edge operator is deployed within the **c8yedge** namespace. If you wish to install the Edge operator and Edge in a different namespace, you can specify it using the **-n** option in the installation script.

Run the following command to follow the logs for the Edge operator pod:

```
kubectll logs -f -n c8yedge deployment/c8yedge-operator-controller-manager manager
```

INFO

Substitute the namespace name **c8yedge** in the command above with the namespace name where you have installed the Edge operator.

INSTALLING THE EDGE OPERATOR (OFFLINE)

Frequently, portions of a data center might not have access to the Internet, even via proxy servers. You can still install Edge in such an environment, but you must make the required software, Helm Charts and Docker images, available to the disconnected environment through an [Open Container Initiative](#) (OCI) compliant private registry.

To enable this, you need to have an OCI compliant registry available in the network which is accessible to the Kubernetes cluster in which you intend to install Edge. You would also need a workstation that has full internet access, to pull the required software from the [Cumulocity registry](#) and push them into the private registry installed or available in the restricted network.

INSTALLING A PRIVATE REGISTRY

Any OCI compliant registry can be used as a private registry, however, the Edge installation is tested with [Harbor](#) and [Nexus Repository OSS](#).

Refer to [Harbor Installation and Configuration](#) for installing Harbor and [Nexus Installation and Upgrades](#) for installing Nexus.

After installing and configuring a private registry, ensure that all the machines (the workstation and the Kubernetes cluster nodes) which need access to the private registry can resolve its domain or host and trust the private registry's certificate (if it is configured with a self-signed certificate).

UPDATE /ETC/HOSTS TO RESOLVE THE DOMAIN

Run the below commands to update the **/etc/hosts** file on every machine (the workstation and the Kubernetes cluster nodes) which needs access to the private registry can resolve its domain or host:

```
PRIVATE_REGISTRY_HOSTNAME="<PRIVATE-REGISTRY-HOSTNAME>" # Change it with your private registry's domain or hostname
PRIVATE_REGISTRY_IP_ADDRESS="<PRIVATE-REGISTRY-IP-ADDRESS>" # Change it with your private registry's IP Address

# Update /etc/hosts to resolve the Harbor domain
echo "${PRIVATE_REGISTRY_IP_ADDRESS} ${PRIVATE_REGISTRY_HOSTNAME}" | sudo tee -a /etc/hosts
```

UPDATE COREDNS CONFIGURATION

Run the commands below to modify the CoreDNS configuration of the Kubernetes cluster to enable resolution of the private registry's domain or host:

```
PRIVATE_REGISTRY_HOSTNAME=""<PRIVATE-REGISTRY-HOSTNAME>" # Change it with your private registry's domain or hostname
PRIVATE_REGISTRY_IP_ADDRESS=""<PRIVATE-REGISTRY-IP-ADDRESS>" # Change it with your private registry's IP Address

# Retrieve the existing NodeHosts value
EXISTING_NODEHOSTS=$(kubectl get configmap coredns -n kube-system -o jsonpath='{.data.NodeHosts}')
EXISTING_NODEHOSTS=$(echo -n "${EXISTING_NODEHOSTS}" | sed ':a;N;$!ba;s/\n/g')

# Append the new domain and IP address to the existing NodeHosts value
UPDATED_NODEHOSTS=$(echo "${EXISTING_NODEHOSTS}\n${PRIVATE_REGISTRY_IP_ADDRESS} ${PRIVATE_REGISTRY_HOSTNAME}")

# Patch the CoreDNS ConfigMap with the updated NodeHosts value
kubectl patch configmap coredns -n kube-system --type merge -p '{"data":{"NodeHosts":"${UPDATED_NODEHOSTS}"}}'
```

TRUST THE PRIVATE REGISTRY'S CERTIFICATE

Run the below commands to trust the private registry's certificate (if it is configured with a self-signed certificate), on every machine (the workstation and the Kubernetes cluster nodes) which needs access to the private registry including the Kubernetes cluster nodes:

```
sudo sh -c '
PRIVATE_REGISTRY_HOST=""<PRIVATE-REGISTRY-HOSTNAME>:<PRIVATE-REGISTRY-PORT>" # Change it with your private registry domain or
hostname:port or ip-address:port

PRIVATE_REGISTRY_CA_CERT=$(echo quit | openssl s_client -showcerts -servername ${PRIVATE_REGISTRY_HOST} -connect
${PRIVATE_REGISTRY_HOST}) && \
if command -v "update-ca-certificates" > /dev/null 2>&1; then
mkdir -p /usr/local/share/ca-certificates
echo "${PRIVATE_REGISTRY_CA_CERT}" > /usr/local/share/ca-certificates/private-registry-ca.crt
update-ca-certificates
elif command -v "update-ca-trust" > /dev/null 2>&1; then
mkdir -p /etc/pki/tls/certs
echo "${PRIVATE_REGISTRY_CA_CERT}" > /etc/pki/tls/certs/private-registry-ca.crt
update-ca-trust extract
fi
'
```

❗ IMPORTANT

You should restart the container runtime and Kubernetes cluster after running the above commands for the changes to take effect. For example, you can restart k3s using `sudo systemctl restart k3s` or `sudo service k3s restart` commands and docker using `sudo systemctl restart docker` or `sudo service docker restart` commands.

DOWNLOAD AND PUBLISH REQUIRED SOFTWARE TO THE PRIVATE REGISTRY

This section outlines the steps to download the required software from the [Cumulocity registry](#) and publish them to the private registry.

For this you need a workstation with full internet access to download the required software from the remote registry and push them into the private registry. Make sure this workstation meets the following prerequisites.

Item	Details
Workstation	A workstation that has full internet access to pull the required software from the remote registry and push them into the private registry.
Python 3	Install Python 3. Refer to Python Setup and Usage for installing Python 3 required to run the registry sync script.

Item	Details
Docker CLI	Install <code>docker-ce</code> and <code>docker-ce-cli</code> packages. Refer to Installing Docker for installation instructions.
Helm version 3.x	Refer to Installing Helm for the installation instructions.
ORAS CLI version 1.0.0	OCI Registry As Storage (ORAS) CLI is used to publish non-container artifacts to the Harbor registry. Refer to Installing ORAS CLI for installation instructions.

Install registry sync script

To install registry synchronization script, run the commands below:

```
pip install --force-reinstall http://localhost:1313/docs/files/edge-k8s/c8yedge_registry_sync-1018.0.1-py3-none-any.whl
```

Run registry sync script

To download the required software from the [Cumulocity registry](#) and publish them to the private registry, run the command below:

INFO

If your private registry is a Harbor registry, you need to pass an extra option `--target-registry-type=HARBOR` to the instruct the script to create the required projects before publishing the required software to it.

Use `-h` or `--help` option to display the usage details.

```
EDGE_REGISTRY_USER="<<EDGE-REGISTRY-USER>" # Edge registry credentials can be obtained from the Cumulocity logistics team for your
region
EDGE_REGISTRY_PASSWORD="<<EDGE-REGISTRY-PASS>" # Edge registry credentials can be obtained from the Cumulocity logistics team for your
region

PRIVATE_REGISTRY_HOST="<<PRIVATE-REGISTRY-HOSTNAME>:<PRIVATE-REGISTRY-PORT>" # Change it with your private registry domain or
hostname:port or ip-address:port
PRIVATE_REGISTRY_USERNAME="<<PRIVATE-REGISTRY-USER>" # Change it with the credentials to access your private registry
PRIVATE_REGISTRY_PASSWORD="<<PRIVATE-REGISTRY-PASSWORD>" # Change it with the credentials to access your private
registry

c8yedge_registry_sync sync -v 1018.0.1 -sr registry.c8y.io -sru "${EDGE_REGISTRY_USER}" -srp "${EDGE_REGISTRY_PASSWORD}" -tr
"${PRIVATE_REGISTRY_HOST}" -tru "${PRIVATE_REGISTRY_USERNAME}" -trp "${PRIVATE_REGISTRY_PASSWORD}" --dryrun False
```

INFO

To request the Edge registry credentials, [contact product support](#).

UPDATE CUSTOM-ENVIRONMENT-VARIABLES CONFIGMAP

Run the below commands to create or update the custom-environment-variables ConfigMap with key "ca.crt" for the Edge operator to trust the private registry's certificate (if it is configured with a self-signed certificate):

```
EDGE_NAMESPACE=c8yedge # Change namespace name if you want to deploy Edge operator and Edge in a different namespace

PRIVATE_REGISTRY_HOST="<PRIVATE-REGISTRY-HOSTNAME>:<PRIVATE-REGISTRY-PORT>" # Change it with your private registry domain or
hostname:port or ip-address:port

PRIVATE_REGISTRY_CA_CERT=$(echo quit | openssl s_client -showcerts -servername ${PRIVATE_REGISTRY_HOST} -connect
${PRIVATE_REGISTRY_HOST})
mkdir -p /tmp
echo "${PRIVATE_REGISTRY_CA_CERT}" > /tmp/private-registry-ca.crt

# Create/Update custom-environment-variables ConfigMap with key "ca.crt" for the edge operator to trust
kubectl create namespace "${EDGE_NAMESPACE}" --dry-run=client -o yaml | kubectl apply -f -
kubectl create configmap custom-environment-variables -n "${EDGE_NAMESPACE}" --from-file=ca.crt="/tmp/private-registry-ca.crt" --dry-run=client -o
yaml | kubectl apply -f -
```

INSTALLING THE EDGE OPERATOR

Continue with installing the Edge operator by following the instructions in [Installing the Edge operator](#) passing the private registry's host (`-r` option) as `<private-registry-hostname>:<private-registry-port>` and the respective registry credentials when prompted.

INSTALLING EDGE

Before you start the installation, ensure that you have fulfilled the [prerequisites](#) and configured the storage as described in [Configuring storage](#).

Download and edit the Edge CR ([c8yedge.yaml](#)), before applying it to your Kubernetes cluster by running the command below:

```
kubectl apply -f c8yedge.yaml
```

For more information about the structure and configuration options available in the Edge CR, see [Edge Custom Resource](#).

VERIFYING THE EDGE INSTALLATION

To monitor the installation progress, run the command below:

```
kubectl describe edge c8yedge -n c8yedge
```

This command allows you to view the details about the installation of `c8yedge` in the `c8yedge` namespace.

INFO

Substitute the Edge name and namespace name, which is currently `c8yedge` in the command, with the specific Edge name and namespace name you have specified in your Edge CR.

You can also follow the events raised for the Edge CR by running the command below:

```
kubectl get events -n c8yedge --field-selector involvedObject.name=c8yedge --watch
```

The **Events** section in the output of the `describe edge` command specifies the installation progress and the **Status** section displays the generation of the Edge CR which is being installed and its current state. Once the installation succeeds, the **Status** section also displays the generation of the CR which is deployed, Edge version, last deployed time/age, validation warnings, if any and some help commands

for downloading the diagnostic logs, extracting the Root CA of the Edge operator generated TLS/SSL certificates.

A sample status output:

```
Name:      c8yedge
Namespace: c8yedge
Kind:      CumulocityIoTEdge

Metadata:
  Creation Timestamp: 2023-08-11T00:00:01Z
  Generation:        1

Spec:
  Version:      1018.0.1
  License Key:   *****
  Company:      IoT Company
  Domain:       myown.iot.com
  Email:        myown@iot.com
  ....
  ....

Status:
  Deployed Generation: 1
  Last Deployed Time: 2023-08-11T00:15:00Z
  State:           Ready
  Version:         1018.0.1-XXXX

Help Commands:
  Download Logs:
FILE_NAME="edge-diagnostic-archive-$(date +%Y%m%d%H%M%S).tar.gz" && \
kubect exec -n edge-sample-logging logging-fluentd-0 -c fluentd -- tar -czvf /var/log/$FILE_NAME /var/log/edge && \
kubectl cp edge-sample-logging/logging-fluentd-0:/var/log/$FILE_NAME -c fluentd ./FILE_NAME && \
kubectl exec -n edge-sample-logging logging-fluentd-0 -c fluentd -- rm /var/log/$FILE_NAME
```

A sample set of installation events:

```
Events:
Type Reason      Age From Message
----
Normal Validating    15m cumulocityiotedge validating
Normal ValidationPassed 15m cumulocityiotedge validation passed
Normal Installing    15m cumulocityiotedge installing
.....
Normal Installing    12m cumulocityiotedge finished installing mongo server
.....
Normal Installing    8m cumulocityiotedge finished installing core
.....
Normal Installing    5m cumulocityiotedge finished installing and updating microservices
.....
Normal Installing    2m cumulocityiotedge finished installing thin-edge
.....
Normal Ready         1m cumulocityiotedge Cumulocity Edge installation is complete, and it's now running version 1018.0.1-XXXX
```

Before you continue, wait for the Edge CR status to reach the **Ready** state.

ACCESSING EDGE

Before you can access Edge, you must first get the external IP address. The Edge operator creates a load balancer service named **cumulocity-core**, which receives an external IP. Clients outside of the cluster can access the Edge through this external IP.

ASSIGNING AN EXTERNAL IP

To get the external IP to access Edge, run the command below:

```
kubectl get service cumulocity-core -n c8yedge
```

INFO

Substitute the namespace name *c8yedge* in the command above with the specific namespace name you have specified in your Edge CR.

Sample output of the `kubectl get service` command:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
cumulocity-core	LoadBalancer	X.X.X.X **REDACTED	X.X.X.X **REDACTED	443:31342/TCP,1883:32751/TCP,8883:32270/TCP

Sometimes the external IP displays as `<pending>` or `<none>`. The IP assignment process is dependent on the Kubernetes hosting environment. An external load balancer in the hosting environment handles the IP allocation and any other configurations necessary to route the external traffic to the Kubernetes service. Most on-premise Kubernetes clusters do not have external load balancers that can dynamically allocate IPs. The most common solution is to manually assign an external IP to the service. This can be done in the service's YAML configuration. You can use the following command to manually assign an external IP to the `cumulocity-core` service (replace `<EXTERNAL-IP>` in the command below with the IP address you want to assign).

```
kubectl patch service cumulocity-core -n c8yedge -p '{"spec":{"type": "LoadBalancer", "externalIPs":["<EXTERNAL-IP>"]}}'
```

INFO

Substitute the namespace name *c8yedge* in the command above with the specific namespace name you have specified in your Edge CR.

INFO

When manually assigning the external IP, see the following Kubernetes API documentation:

"These IPs are not managed by Kubernetes. The user is responsible for ensuring that traffic arrives at a node with this IP."

You can access Edge using a domain name in a web browser.

ACCESSING EDGE USING THE DOMAIN NAME

Access Edge using the domain name configured as part of the installation. There are two ways of configuring the accessibility with the domain names:

- Add an entry of the domain name and IP address mapping in the DNS servers.
For example, if your domain name is **myown.iot.com**, add an entry for both **myown.iot.com** and **management.myown.iot.com**.
- Alternatively, [Add the alias](#) to access Edge through the domain name provided during installation. This must be performed on each client host on which Edge is accessed.

The first option is always preferable so that Edge is accessible over LAN.

Adding the alias

On Linux machines, add the following entry to `/etc/hosts`:

```
<IP address> <domain_name>
<IP address> management.<domain_name>
```

Use the external IP address fetched by running the command `kubectl get service` in the previous section.

On Windows machines, add the same entry to `C:\Windows\System32\drivers\etc\hosts`.

Ping the `<domain_name>` to verify it.

```
ping <domain_name>
ping management.<domain_name>
```

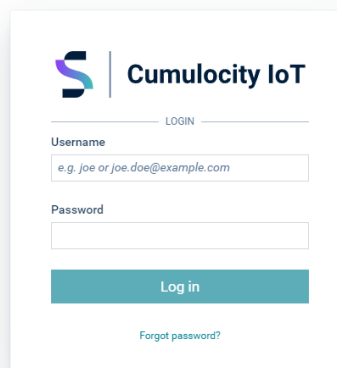
If the ping is successful, the DNS resolution is working properly.

To access Edge

To access Edge, enter one of the following URLs in the browser:

- For the “edge” tenant, use the URL `https://<domain_name>`.
- For the Management tenant, use the URL `https://management.<domain_name>`.

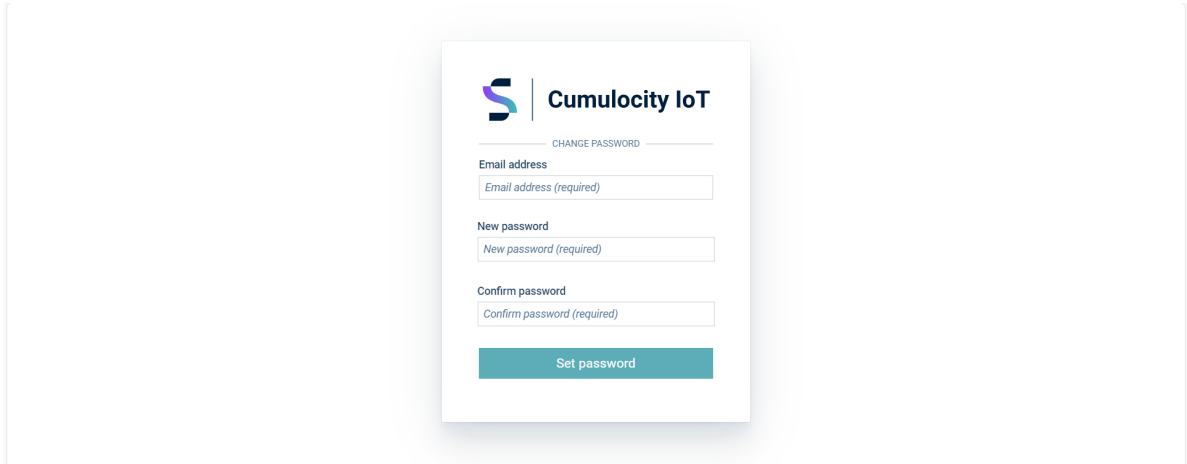
This will bring up the below login screen. Enter the default credentials username “admin” and password “admin-pass” to log in in to both the “edge” tenant and the Management tenant.



On the first login, you see the dialog window below, forcing you to change the password. The email address to change the password is the one you specified in the Cumulocity Edge CR (or `myown@iot.com` if you followed the Quickstart installation steps). Alternatively, run the following command to retrieve the email address:

```
kubectl get edge c8yedge -n c8yedge -o jsonpath='{.spec.email}' && echo
```

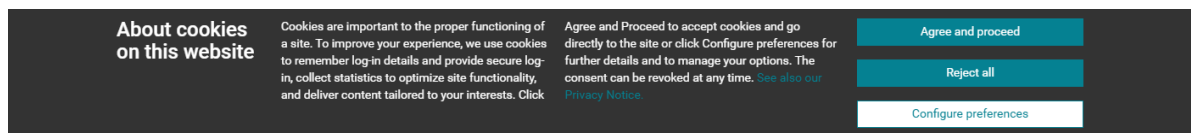
Substitute the Edge name and namespace name, which is currently **c8yedge** in the command, with the specific Edge name and namespace name you have specified in your Edge CR.



! IMPORTANT

After a successful deployment, it is crucial to access both the Management tenant and the “edge” tenant and change their respective admin credentials.

If you are logging in for the first time, you will see a cookie banner at the bottom of the login screen:



i INFO

The cookie banner is turned on by default. This feature can be configured. For more information, see [Branding](#).

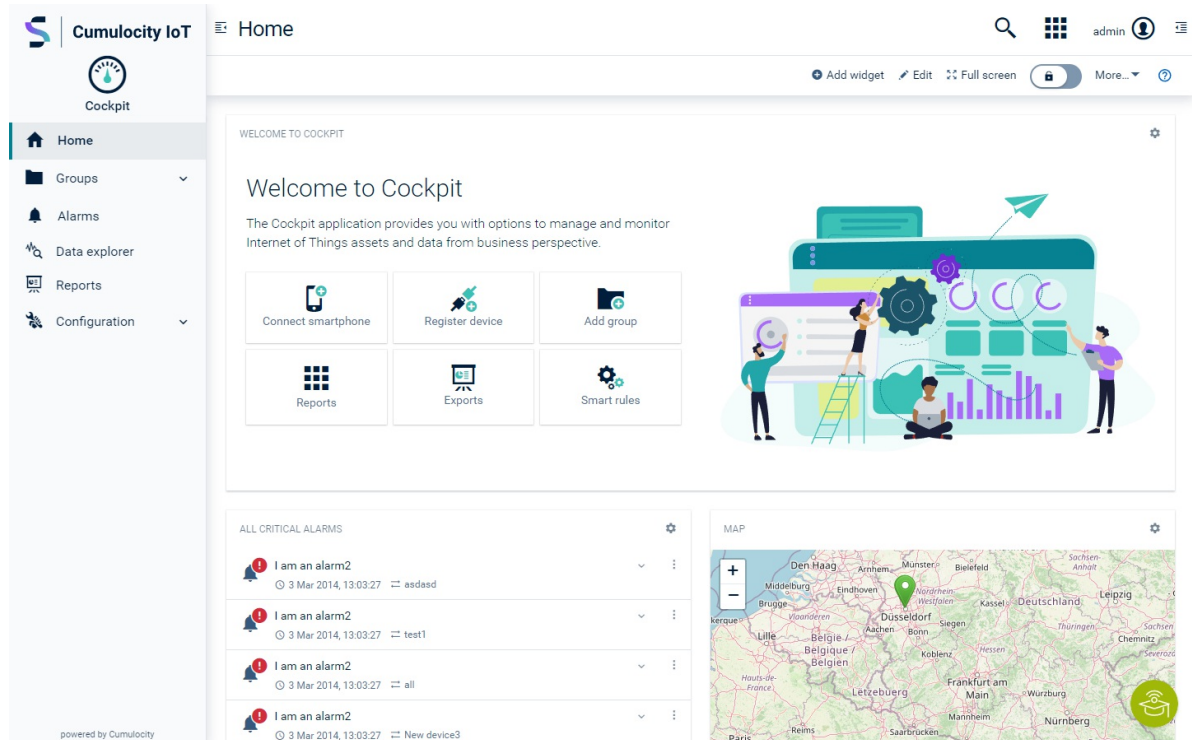
- Click **Agree and Proceed** to accept the default cookie settings (required and functional cookies enabled).
- Click **Reject all** to reject all of the default cookie settings.
- Click **Preferences** to select your individual cookie preferences:
 - **Required** - Required to enable core site functionality. They perform a task or operation without which a site's functionality would not be possible. Required cookies cannot be disabled.
 - **Functional** - Used to track site usage and to process personal data to measure and improve usability and performance. Functional cookies must be actively enabled.
- Click **See also our Privacy Notice** to open the [Cumulocity privacy statement](#) with details on the Cumulocity privacy policy.

i INFO

If you have enabled functional cookies you can opt out of the product experience tracking later on via the **User settings** dialog, see [User options and settings](#).

Select the **Remember me** checkbox if you want the browser to remember your credentials, so that you do not have to enter them again when opening the application the next time. This is especially convenient if you frequently switch between Cumulocity applications, as Edge requests you to authenticate each time when starting an application. You can make the browser “forget” your credentials by explicitly logging out.

Finally, click **Login** to enter Edge. Initially, you will be taken to the [Cockpit](#) application, if not configured differently. For further information about the Cumulocity standard applications see [Available applications](#).



To explicitly log out, click the **User** button at the right of the top bar, then select **Logout** from the context menu.

INFO

The maximum number of failed logins (due to invalid credentials), after which a user is locked, can be configured by the Management tenant on platform level. Contact your Operations team for further support. The default value is 100.

HOW TO RESET OR CHANGE YOUR PASSWORD

To reset your password, you must first configure the “reset password” template and email server settings in Edge. For information about configuring the email server, see [Configuring the email server](#).

For information about changing the password, see [To change your password](#).

HOW TO ACCESS PAGES USING URLS

For information about accessing pages using the URLs, see [URL](#).

ACCESSING LOGS

The Edge operator deploys and configures a Fluent Bit daemonset on the Kubernetes node to collect the container and application logs from the node file system. Fluent Bit queries the Kubernetes API, enriches the logs with metadata about the pods (in the Edge namespace), and transfers both the logs and metadata to Fluentd. Fluentd receives, filters, and persists the logs in the persistent volume claim configured for logging.

To download the diagnostic log archive, run the command below. It generates a file named *c8yedge-logs-{current date}.tar.gz* in the current directory.

```
kubect! get edge c8yedge -n c8yedge --output jsonpath='{.status.helpCommands.downloadLogs}' | sh
```

INFO

Substitute the Edge name and namespace name *c8yedge* in the command above with the specific Edge name and namespace name you have specified in your Edge CR.

Download the log archives remotely from your cloud tenant. For more information, see [Downloading diagnostics remotely](#).

EDGE CUSTOM RESOURCE

This section describes the structure and configuration options available in the Cumulocity Edge CR. Here is the Edge CR template ([c8yedge.yaml](#)) that you can edit and apply to your Kubernetes cluster to install or update Cumulocity Edge.

CONFIGURATION

The initial part of the CR defines the CRD details, and the CR name and namespace, which referred to in this document as **EDGE-CR-NAME** and **EDGE-CR-NAMESPACE**.

```
apiVersion: edge.cumulocity.com/v1
kind: CumulocityIoTEdge
metadata:
  name: <EDGE-CR-NAME>
  namespace: <EDGE-CR-NAMESPACE>
```

SPECIFICATION

This section defines the Edge deployment's configurations.

Field	Required	Type	Default	Description
version	Yes	String		Edge version to deploy. For example, 1018.0.0 for 10.18 and 1018.0.1 for a fix-1 of 10.18.
domain	Yes	String		A fully qualified domain name. For example, <i>myown.iot.com</i> . Here, you must have the Edge license for the domain name <i>iot.com</i> or <i>myown.iot.com</i> .
licenseKey	Yes	String		Edge license key you received for the domain. To request the license file for Edge, contact product support In the email, you must include - Your company name, under which the license has been bought - The domain name (for example, <i>myown.iot.com</i>), where Edge will be reachable For more information, see Domain name validation for Edge license key generation .
company	Yes	String		Name of the "edge" tenant, for example, the company's name.

Field	Required	Type	Default	Description
tlsSecretName	No	String	The Edge operator generates and assigns self-signed TLS/SSL private key and certificates.	Name of the Kubernetes secret containing the TLS/SSL private key and certificates for the domain name specified in the <code>spec.domain</code> field. This secret must contain two keys: - tls.key: TLS/SSL private key in the PEM format. - tls.crt: The TLS/SSL certificate chain associated with the private key in PEM format. It's essential to ensure the certificates are arranged in the correct sequence for TLS/SSL validation to succeed. The proper order of the certificate chain is: - End-entity certificate: This is the TLS/SSL certificate issued to your domain or server, sometimes referred to as the "leaf" or "server" certificate. - Intermediate certificate(s): These certificates link your end-entity certificate to the trusted root certificate. If there are multiple intermediate certificates, they must be ordered correctly as well. - Root CA certificate: This is the certificate for the Certificate Authority (CA) that is trusted by browsers and other clients. It's generally included last in the chain. For more information, see TLS/SSL Secret. Info: The Edge operator retrieves this secret from the <code>EDGE-CR-NAMESPACE</code>. Ensure that this secret is created before initiating the Edge deployment or update process.
email	Yes	String		Email used for the admin user.
cloudTenant	No	Structure		Cumulocity cloud tenant details to configure and manage Edge remotely. For more information, see Cloud Tenant .

Field	Required	Type	Default	Description
storageClassName	No	String		The Edge operator requests three PVCs, as outlined below. - 75 GB, PVC named <code>mongod-data-edge-db-rs0-0</code> made by MongoDB server for persisting application data. 75 GB is the default, and its value can be configured through the Edge CR field <code>spec.mongodb.resources.requests.storage</code>. - 10 GB, PVC named <code>microservices-registry-data</code> made by the private registry for persisting microservice images. - 5 GB, PVC named <code>edge-logs</code> made by the Edge logging component for persisting application and system logs. Each of these PVCs utilizes the StorageClass if specified within the <code>storageClassName</code> field of the Edge CR. - In case you omit the <code>storageClassName</code>, the Edge operator requests PVCs without a StorageClass, thereby instructing Kubernetes to utilize the default StorageClass configured in the cluster. - If you explicitly specify an empty StorageClass as <code>""</code>, the Edge operator requests PVCs with an empty StorageClass, thereby instructing Kubernetes to carry out static provisioning. - Finally, if you specify the name of an existing StorageClass for which dynamic provisioning is enabled, the Operator requests PVCs with that class name, thereby instructing Kubernetes to utilize dynamic provisioning according to the specified class. Info: This value is used only during the Edge installation and can't be changed for existing installations.
core	No	Structure		Cumulocity platform configurations. For more information, see Cumulocity Core configurations.
mongodb	Yes	Structure		Configurations needed to deploy the MongoDB server managed by the Edge operator or connect to an external one. For more information, see MongoDB.

Field	Required	Type	Default	Description
microservices	No	Array of Structure	The Edge operator deploys all the default Cumulocity microservices, which include the Apama, Smart Rules, OPCUA Management Server microservices.	Specify resources to allocate to each of the default Cumulocity microservices deployed. For more information, see Microservices .

TLS/SSL SECRET

Specifies the Kubernetes secret containing the TLS/SSL private key and certificate chain. The Edge operator retrieves this secret from the `EDGE-CR-NAMESPACE`. Ensure that the secret is created before initiating the Edge deployment or update process.

This secret should contain the fields described in the table below.

Field	Required	Type	Default	Description
tls.key	Yes	String		TLS/SSL private key in the PEM format.

Field	Required	Type	Default	Description
tls.crt	Yes	String		The TLS/SSL certificate chain associated with the private key in PEM format. It's essential to ensure the certificates are arranged in the correct sequence for TLS/SSL validation to succeed. The proper order of the certificate chain is: - End-entity certificate: This is the TLS/SSL certificate issued to your domain or server, sometimes referred to as the "leaf" or "server" certificate. - Intermediate certificate(s): These certificates link your end-entity certificate to the trusted root certificate. If there are multiple intermediate certificates, they must be ordered correctly as well. - Root CA certificate: This is the certificate for the Certificate Authority (CA) that is trusted by browsers and other clients. It's generally included last in the chain.

CLOUD TENANT

Edge can be managed, configured, and monitored remotely through a Cumulocity cloud tenant. You can control and troubleshoot your Edge deployments remotely.

To enable this, first register Edge as a device in the Cumulocity cloud tenant. You can register Edge by providing the Cumulocity cloud tenant URI, and optionally, TLS/SSL key and certificate chain with which Edge connects to cloud through MQTT protocol using a X.509 certificate for authentication. If you do not provide the TLS/SSL key and the certificate chain, the Edge operator uses an internally generated TLS/SSL key and certificate for identifying Edge as a device in the cloud tenant. For completing the registration process, sign into your cloud tenant and follow the steps described in [Managing trusted certificates](#) to add the Certificate Authority (CA) certificate to the trusted certificate list of your tenant. For more information, see [Device certificates](#).

In case you let the Edge use the internally generated TLS/SSL key and certificates, you can download the CA certificate by using the command below:

```
kubectl get edge c8yedge -n c8yedge --output jsonpath='{.status.helpCommands.fetchGeneratedCACrt}' | sh
```

INFO

Substitute the Edge name and namespace name `c8yedge` in the command above with the specific Edge name and namespace name you have specified in your Edge CR.

Once registered, you can access your Edge remotely as a device, monitor its metrics, upgrade its version and collect diagnostic data remotely.

Field	Required	Type	Default	Description
domain	Yes	String		Cumulocity cloud tenant domain. For example, <code><tenantid>.cumulocity.com</code>

Field	Required	Type	Default	Description
tlsSecretName	No	string	The Edge operator generates and assigns self-signed TLS/SSL private key and certificates.	Name of the Kubernetes secret containing the TLS/SSL private key and certificates with which Edge connects to the cloud through MQTT protocol using a X.509 certificate for authentication. This secret must contain two keys: - tls.key: TLS/SSL private key in the PEM format. - tls.crt: The TLS/SSL certificate chain associated with the private key in PEM format. It's essential to ensure the certificates are arranged in the correct sequence for TLS/SSL validation to succeed. The proper order of the certificate chain is: - End-entity certificate: This is the TLS/SSL certificate issued to your domain or server, sometimes referred to as the "leaf" or "server" certificate. - Intermediate certificate(s): These certificates link your end-entity certificate to the trusted root certificate. If there are multiple intermediate certificates, they must be ordered correctly as well. - Root CA certificate: This is the certificate for the Certificate Authority (CA) that is trusted by browsers and other clients. It's generally included last in the chain. Info: You can also reuse the secret name provided in the <code>spec.tlsSecretName</code> provided that the TLS/SSL certificate it references is issued by an intermediate Certificate Authority (CA) within your organization and can be added to the trusted certificate list of your Cumulocity cloud tenant. For more information, see TLS/SSL Secret. Info: The Edge operator retrieves this secret from the <code>EDGE-CR-NAMESPACE</code>. Ensure that this secret is created before initiating the Edge deployment or update process.

CUMULOCITY CORE

The core specification specifies the fields for configuring the Cumulocity core node and its resource limits.

Field	Required	Type	Default	Description
-------	----------	------	---------	-------------

Field	Required	Type	Default	Description
resources.limits	Yes	Structure	Defaults to CPU Limit: 3000m Memory Limit: 6GB	Specify resource limits for the Cumulocity Core container. For more information, see Resource limits specification .

MONGODB

This field is necessary for one or more of the following reasons:

- To specify the MongoDB admin credentials.
- To configure an externally hosted MongoDB server.
- To specify resource limits for the MongoDB server containers deployed by the Edge operator.

Field	Required	Type	Default	Description
credentialsSecretName	No	String	Defaults to <code>databaseAdmin</code> and <code>admin-pass</code> as the database admin user and password for the Edge operator managed MongoDB or fails with validation error for the externally hosted MongoDB server. Info: For the MongoDB managed by the Edge operator, it is recommended to provide the database admin credentials secret, rather than relying on the default credentials assigned by the Edge operator.	Name of the Kubernetes Secret containing the database admin credentials with which the Edge operator managed MongoDB must be configured or the database admin credentials of the externally hosted MongoDB server. For more information, see MongoDB Credentials Secret . Info: The Edge operator retrieves this secret from the <code>EDGE-CR-NAMESPACE</code> . Ensure that this secret is created before initiating the Edge deployment or update process.

Field	Required	Type	Default	Description
connectionString	No	String	If not provided, the Edge operator installs a single node MongoDB server and configures it with the admin credentials provided in <code>spec.mongodb.credentialsSecretName</code>	Connection string of the externally hosted MongoDB server. URI format: <pre>mongodb://host1[:port1][,...hostN[:portN]]</pre> Info: If you do not provide this value, the Edge operator installs a single node MongoDB server. Once Edge is installed and configured to use the MongoDB managed by the Edge operator, you cannot provide the <code>connectionString</code> to use an externally hosted MongoDB.
tlsSecretName	No	String	Not required if the TLS/SSL key and certificates are issued by a trusted public CA.	Secret for supplying the Certificate Authority (CA) certificate that issued the TLS/SSL key and certificates used to configure the TLS for the externally hosted MongoDB server. This allows the Edge operator to establish trust with the server. For more information, see Externally hosted MongoDB TLS/SSL Secret. Info: The Edge operator retrieves this secret from the <code>EDGE-CR-NAMESPACE</code>. Ensure that this secret is created before initiating the Edge deployment or update process.
resources.limits	Yes	Structure	Defaults to CPU Limit: 3000m Memory Limit: 6GB	Specify resource limits for the MongoDB server pod. For more information, see Resource limits specification .
resources.requests	No	Structure	Defaults to 75 GB	Specify the size of the Persistent Volume Claim (PVC) named <code>mongod-data-edge-db-rs0-0</code> made by MongoDB server for persisting application data. For more information, see MongoDB storage size .

RESOURCE LIMITS SPECIFICATION

Structure for specifying the resource limits for the containers deployed by the Edge operator.

Field	Required	Type	Default	Description
cpu	No	String		Specific CPU limit in CPU units. For example, 1000m or 1M
memory	No	String		Specific memory limit in memory units. For example, 1000M or 1G

MONGODB CREDENTIALS SECRET

Specifies the Kubernetes secret containing the admin credentials with which the MongoDB managed by the Edge operator must be configured or the admin credentials of the externally hosted MongoDB server. The Edge operator retrieves this secret from the namespace `EDGE-CR-NAMESPACE`. It is important that this secret is created before initiating the Edge deployment or update process.

This secret should contain the fields described in the table below:

Field	Required	Type	Default	Description
MONGODB_DATABASE_ADMIN_USER	Yes	String		Database admin username with which the MongoDB managed by the Edge operator or the username of the externally hosted MongoDB server is configured.
MONGODB_DATABASE_ADMIN_PASSWORD	Yes	String		Database admin password with which MongoDB managed by the Edge operator or the password for the externally hosted MongoDB server is configured.
MONGODB_USER_ADMIN_USER	No	String	userAdmin	Only used when MongoDB is deployed and managed by the Edge operator.
MONGODB_USER_ADMIN_PASSWORD	No	String	Password provided in the field <code>MONGODB_DATABASE_ADMIN_PASSWORD</code>	Only used when MongoDB is deployed and managed by the Edge operator.
MONGODB_CLUSTER_ADMIN_USER	No	String	clusterAdmin	Only used when MongoDB is deployed and managed by the Edge operator.
MONGODB_CLUSTER_ADMIN_PASSWORD	No	String	Password provided in the field <code>MONGODB_DATABASE_ADMIN_PASSWORD</code>	Only used when MongoDB is deployed and managed by the Edge operator.
MONGODB_CLUSTER_MONITOR_USER	No	String	clusterMonitor	Only used when MongoDB is deployed and managed by the Edge operator.

Field	Required	Type	Default	Description
MONGODB_CLUSTER_MONITOR_PASSWORD	No	String	Password provided in the field MONGODB_DATABASES E_ADMIN_PASSWORD	Only used when MongoDB is deployed and managed by the Edge operator.
MONGODB_BACKUP_USE	No	String	backup	Only used when MongoDB is deployed and managed by the Edge operator.
MONGODB_BACKUP_PASSWORD	No	String	Password provided in the field MONGODB_DATABASES E_ADMIN_PASSWORD	Only used when MongoDB is deployed and managed by the Edge operator.

EXTERNALLY HOSTED MONGODB TLS/SSL SECRET

Field	Required	Type	Default	Description
ca.crt	No	String		Certificate Authority (CA) certificate in PEM format. Only required if the externally hosted MongoDB is TLS/SSL enabled with a self-signed certificate or a certificate not issued by a publicly trusted CA.

MONGODB STORAGE SIZE

Field	Required	Type	Default	Description
storage	No	string	Default to 75 GB	Specify the PVC storage. Info: Once Edge is installed, you can only increase this value, but cannot reduce.

MICROSERVICES

The microservice specification allows specifying resources to allocate to a default microservice, which includes the Apama, Smart Rules, OPCUA Management Server and Device Simulator microservices.

Field	Required	Type	Default	Description
-------	----------	------	---------	-------------

Field	Required	Type	Default	Description
name	Yes	String		The name of the Cumulocity microservice. The allowed values are apama-ctrl, smartrule, ssl-management-server, device-simulator, and opcua-mgmt-service
resources.limits	No	Structure	Defaults to CPU Limit: 1000m Memory Limit: 1 GB	Specify resource limits for the Cumulocity microservice container. For more information, see Resource limits specification .

CONNECTING EDGE TO THE CLOUD

Edge can be managed, configured, and monitored remotely through a Cumulocity cloud tenant. You can control and troubleshoot your instance of Edge deployments remotely.

REGISTERING EDGE IN THE CLOUD TENANT

To manage, configure, and monitor remotely through a Cumulocity cloud tenant, you must first register Edge as a device in the Cumulocity cloud tenant. You can register Edge by providing the Cumulocity cloud tenant URI, and optionally, TLS/SSL key and certificate chain with which Edge connects to cloud through MQTT protocol using an X.509 certificate for authentication. If you do not provide the TLS/SSL key and the certificate chain, the Edge operator uses an internally generated TLS/SSL key and certificate for identifying Edge as a device to the cloud tenant. For completing the registration process, you must sign into your cloud tenant and follow the steps described in [Managing trusted certificates](#) to add the Certificate Authority (CA) certificate to the trusted certificate list of your tenant.

In case you let Edge use the internally generated TLS/SSL key and certificates, you can download the CA certificate by using the following command:

```
kubectll get edge c8yedge -n c8yedge --output jsonpath='{.status.helpCommands.fetchGeneratedCACrt}' | sh
```

INFO

Substitute the namespace name `c8yedge` in the command above with the specific namespace name you have specified in your Edge CR.

Once registered, the Edge deployment appears as a device named after the domain name of your Edge instance. For example, if you have configured your domain name as **myown.iot.com**, the Edge deployment appears as **myown.iot.com**.

UPGRADING EDGE REMOTELY

You can remotely update your instance of Edge using the Cumulocity's firmware update feature. This process requires you to upload a YAML file specifying the Edge version to your tenant account. For more information about the firmware feature, see [Managing firmware](#).

Update the [sample version file](#) to specify the Edge version follow the steps below to upgrade:

1. Log in to your Cumulocity tenant account and go to the Device Management application.
2. In the **Firmware repository** page, click **Add firmware** and provide a name for the firmware, add a description and its version (all required).

The screenshot shows the Cumulocity IoT interface. On the left is a sidebar with 'Device management' and 'Management' sections. The 'Firmware repository' option is selected under 'Management'. The main area is titled 'Firmware repository' and contains a table with the following data:

Firmware	Description	Device type	Versions
Edge Upgrades	upgrade to latest version	Undefined	2 versions
child-iot-linux	No description	thin-edge.io-child	2 versions
test	Test upgrade	Undefined	3 versions
thick edge versions	edge versions	Undefined	2 versions

Optionally, you can define the device type filter when adding a new firmware. For example, `c8y_EdgeAgent`.

3. Select the **Provide a file path** option to specify an HTTPS URL of a server from where the version file can be downloaded.

Click **Add firmware**. The firmware object appears in the firmware list.

4. Click **All devices** in the **Devices** menu, select your instance of Edge from the device list.
5. Click **Firmware**. The **Firmware** tab shows the current Edge version.

The screenshot shows the Cumulocity IoT interface for a specific device instance 'myown.iot.com'. The 'Firmware' tab is selected in the left sidebar. The main area displays the 'Current firmware' section with the following information:

Current firmware
Edge VERSION 1017.0.0
Replace firmware

You can also see the current Edge version in the **Info** tab.

The screenshot shows the 'Info' tab of the IoT Edge Agent configuration. The left sidebar contains navigation links: Info, Measurements, Alarms, Control, Firmware, Diagnostics, Availability, Events, and Identity. The main content area displays 'DEVICE DATA' with the following details:

ID	3529341190
Name	myown.iot.com
Type	c8y_EdgeAgent
Last updated	2023-08-23T15:02:32.888Z
Creation time	2023-08-23T12:48:54.796Z

Below the device data, the 'Firmware' section is highlighted with an orange box, showing the following details:

Name	Edge
Version	1017.0.0

6. Click **Replace firmware**.
7. Select the firmware that you just uploaded and click **Install**.

The screenshot shows the 'Edge Upgrades' section with the text 'upgrade to latest version' and a '2 versions' link. Below this, there are two radio button options for selecting a firmware version:

- ☒ 1017.0.1
- ☐ 1017.0.0

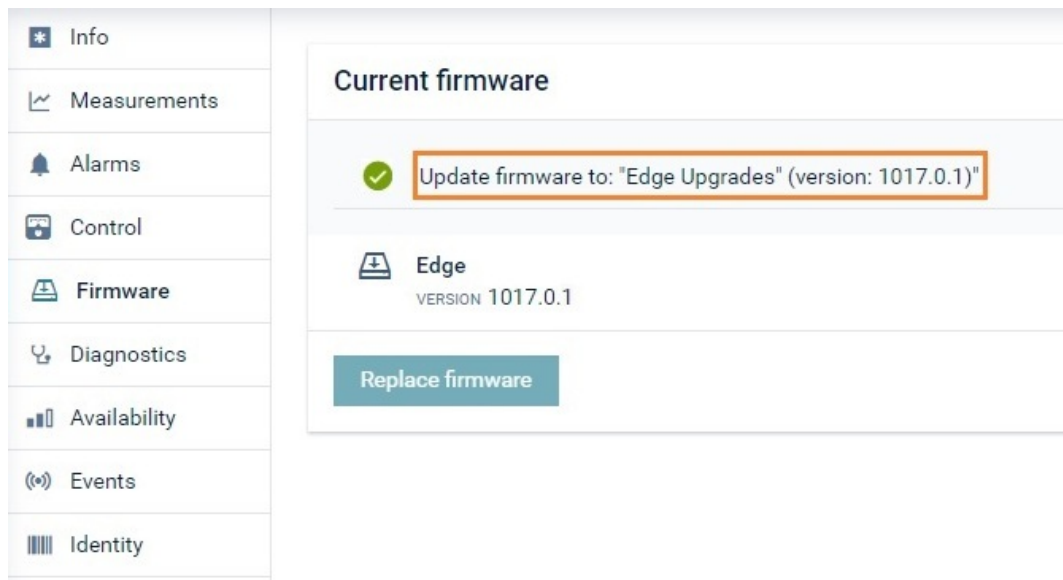
- To check the status of the update, hover over the refresh icon  as shown in the figure below:

The screenshot shows the 'Firmware' tab of the IoT Edge Agent configuration. The left sidebar is the same as in the previous screenshot. The main content area displays 'Current firmware' with the following details:

EXECUTING firmware to: "Edge Upgrades" (version: 1017.0.1)

Below this, there is a section for the 'Edge' firmware with the version '1017.0.0'. At the bottom of this section, there is a button labeled 'Replace firmware'.

- To check the details of the update, click the text outlined in red in the figure below:



UPDATING EDGE APPLIANCES USING BULK OPERATIONS

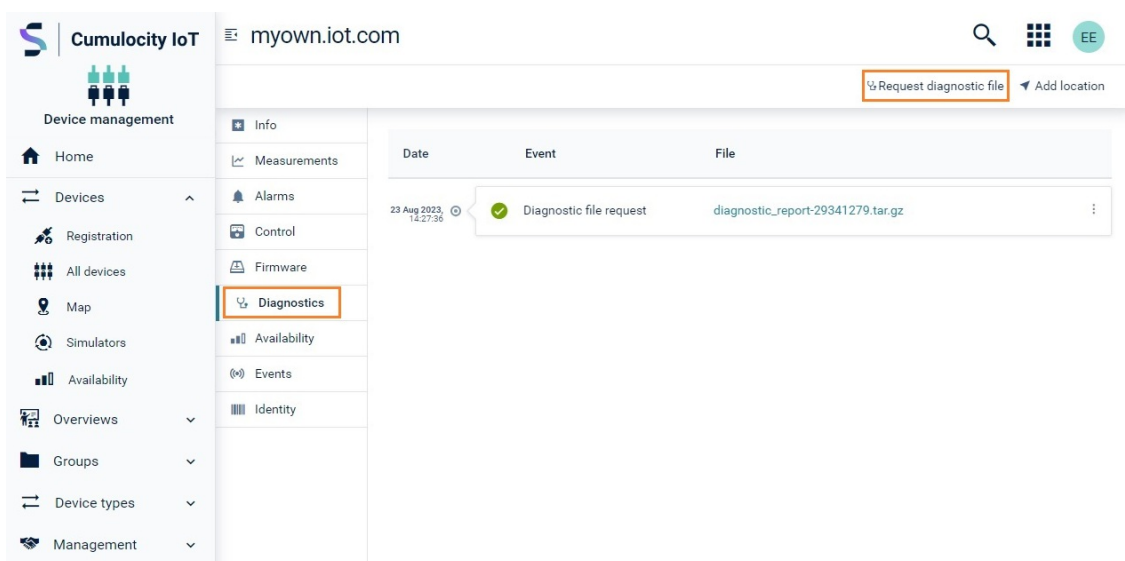
For information about updating Edge using bulk operations, see [Updating Edge using bulk operations](#).

DOWNLOADING DIAGNOSTICS REMOTELY

You can download the diagnostics report remotely from your tenant account after you have registered your instance of Edge in the tenant account.

To download the diagnostics report from your tenant account:

1. Log in to your Cumulocity tenant.
2. Go to the Device Management application.
3. Click **Devices** > **All devices** in the navigator.
4. Select your instance of Edge for which you want to download the diagnostics report.
5. Click **Diagnostics** > **Request diagnostic file**.



6. Click the ZIP file link to download the diagnostic report.

MANAGING SOFTWARE REMOTELY

You can remotely manage the **applications**, **microservices** and **extensions** installed on Edge using the Cumulocity [advanced software management](#) feature. This process requires you to first upload an application, microservice or extension to the software repository before installing them. For more information on uploading items to the software repository, see [To add a new software or software version](#). For more information about the software management feature in general, see [Managing device software](#).

! IMPORTANT

To use the Cumulocity [advanced software management](#) feature, your Cumulocity cloud tenant's subscription plan must include the *advanced-software-mgmt* microservice.

- Applications - You need a Cumulocity application, see [Web SDK](#) for details. Upload the application zip to the software repository with the software type *c8yedge_application*
- Microservices - You need a microservice, see [Microservice SDK](#) for details. Upload the microservice zip to the software repository with the software type *c8yedge_microservice*
- Extensions - You need either a widget or plugin, see [Micro frontends](#) for details. Upload the extension zip to the software repository with the software type *c8yedge_extension*

i INFO

When using the remote software management feature, the manifest for applications, microservices and extensions must contain a *name*, *key*, *version* and *contextPath*. These values are used when installing the software rather than using the name and version from the software repository. The name and version from the manifest are also listed on the devices **Software** tab when installed.

INSTALLING SOFTWARE

1. Navigate to your Edge device's **Software** tab. The **Software** tab lists your installed applications, microservices and extensions. Click **Install software**. The **Install software** dialog lists the software from the software repository which matches Edge's supported types of *c8yedge_application*, *c8yedge_microservice* and *c8yedge_extension*.
2. Select one or multiple software items.
3. Click **Install**.
4. Under **Software changes** on the right, review your planned changes and confirm the software update operation by clicking **Apply changes**.

The install operation will be created and executed on the device. The software installation is completed when the device has installed the software.

i INFO

Applications and microservices will be installed and subscribed to on your 'edge' tenant but extensions will only be installed.

UPDATING SOFTWARE

To update software on your Edge device, hover over the software item which you want to update and click **Update**. Select a version from the list and click **Update** again. The software will be updated with the selected version.

The update operation will be created and executed on the device. The software update is completed when the device has updated the software.

REMOVING SOFTWARE

To remove software on your Edge device, hover over the software item which you want to remove and click the remove icon.

The remove operation will be created and executed on the device. The software removal is completed when the device has removed the software.

DATA EXCHANGE USING DATA BROKER

Data broker lets you upload the data to a Cumulocity tenant account selectively. Note that you must first create a Cumulocity tenant account.

! IMPORTANT

Data broker and Edge connectivity are compatible only with the current version and previous two releases of Cumulocity. Cumulocity has a release every three months.

You can share the following data with the tenant account:

- Devices (and more generically, managed objects)
- Events
- Alarms
- Measurements

! IMPORTANT

Data broker in Edge does not support synchronization of the [operations](#).

To upload the data to a Cumulocity tenant account, you must first create a data connector in the Edge appliance and subscribe this connector in the tenant account.

A data connector describes the subset of the data that you would like to send to a destination tenant. For more information, see [Data connectors](#).

To create a data connector and upload the data to the tenant account, perform the following steps:

1. In your Edge, log in to the "edge" tenant.
2. In your Edge, go to the Administration application. Click **Data broker > Data connectors**.
3. Click **Add data connector** and provide all the information and filters. See [To add a data connector](#).

Note down the security code. This security code will be used to subscribe the connector in the tenant account.

4. Log in to the Cumulocity tenant account. For more information on, see [To access Edge](#)
5. In the tenant account, go to the Administration application. Click **Data broker > Data subscriptions** to subscribe the connector created in your Edge appliance.

Click **Add data subscription** and provide the security code. Click **Submit** and accept the subscription. See [Data subscriptions](#).

You can now navigate to the Device Management application or the Cockpit application. You will find a new "virtual group" with a specific icon showing the forwarded devices. The group will have the same name as your subscription.

For more information about sending and receiving data in Cumulocity, see [Using the data broker](#).

🏠 > Edge on Kubernetes > Connecting Edge to the cloud

MANAGING EDGE

MODIFYING EDGE

For this example, assume that the Edge is deployed using the [c8yedge-sample.yaml](#).

EXAMPLE CONFIGURATION CHANGE

Change the `spec.domain` field to `sample.myown.iot.com` and if required update the `spec.licenseKey` accordingly.

Save the file and use the command below to apply the changes:

```
kubectl apply -f c8yedge-sample.yaml
```

To verify the Edge deployment, see [Verifying the Edge installation](#).

UPGRADING EDGE

The Edge operator follows the recreate update strategy to upgrade the Edge deployment.

Recreating update strategy is an all-or-nothing process that updates all aspects of the system at once with a brief downtime period. The Edge operator selects all the outdated pods and deactivates them at once. Once all old pods are deactivated, the Edge operator creates updated pods for the entire system. Edge is not operational while the old pods are deactivating and until the final updated pod is created.

For this example, assume that Edge is deployed using the [c8yedge-sample.yaml](#).

STARTING THE UPGRADE

To upgrade the Edge deployment, change the `spec.version` field in the Edge CR file to the appropriate version. For example to `1018.0.1`.

Save the file and use the command below to apply the changes:

```
kubectl apply -f c8yedge-sample.yaml
```

To verify the Edge deployment, see [Verifying the Edge installation](#).

UPGRADING EDGE REMOTELY

For information about upgrading Edge remotely, see [Upgrading Edge remotely](#).

RESTARTING THE EDGE OPERATOR

Run this command to restart the Edge operator.

```
kubectl rollout restart deployment -n c8yedge c8yedge-operator-controller-manager
```

INFO

Substitute the namespace name *c8yedge* in the command above with the namespace name where you have installed the Edge operator.

UPGRADING THE EDGE OPERATOR

The Edge operator, once installed, will automatically upgrade when you upgrade Edge to a new version.

UNINSTALLING EDGE

You can uninstall Edge either by:

- Deleting the Edge Custom Resource using the command:

```
kubectl delete edge c8yedge -n c8yedge
```

- Through the [c8yedge-sample.yaml](#) that you used to install Edge.

For example, you can uninstall by using the command:

```
kubectl delete -f c8yedge-sample.yaml
```

INFO

Substitute the namespace name *c8yedge* in the command above with the specific namespace name you have specified in your Edge CR.

UNINSTALLING THE EDGE OPERATOR

Run this command for removing the Edge operator from your Kubernetes cluster.

```
helm uninstall c8yedge-operator -n c8yedge
```

INFO

Substitute the namespace name *c8yedge* in the command above with the namespace name where you have installed the Edge operator.

EDGE OPERATIONS

This section describes the main operating procedures for standard tasks that must be carried out when managing Cumulocity Edge.

MONITORING

Edge on Kubernetes allows for monitoring of the Edge deployment using Prometheus, an open-source project that is used for monitoring application state. See <https://prometheus.io/> for detailed information on Prometheus and how to use it.

The Edge operator exposes a Prometheus-compatible metrics endpoint, <https://<domain>:3443/metrics>, where the domain is the one you specified in the Edge CR (or myown.iot.com if you followed the Quickstart installation steps). You can monitor the recommended list of metrics below, with more available at the above endpoint.

CUMULOCITY CORE METRICS

Metrics related to Cumulocity Core, prefixed by **c8yedge_core**

Metric	Description	Interpretation
c8yedge_core_sag_c8y_process_cpu_usage	CPU usage percentage of Cumulocity Core process.	Key for monitoring the CPU demand of the Cumulocity Core process, helping in identifying high CPU consumption issues.
c8yedge_core_sag_c8y_system_cpu_count	The total count of CPU cores available to the Cumulocity Core system.	Useful for understanding the computing capacity of the system and for scaling considerations.
c8yedge_core_sag_c8y_system_load_average_1m	The 1-minute average system load.	Indicates the immediate demand placed on the system's resources, helping to identify spikes in usage.
c8yedge_core_sag_c8y_process_files_open_files	The number of files currently open by Cumulocity Core process.	Important for monitoring resource utilization and preventing potential exhaustion of file descriptors.

APAMA METRICS

Metrics related to the Apama Microservice, prefixed by **c8yedge_apama**

For more details, see [Monitoring with Prometheus](#) in the Apama documentation.

Metric	Description	Interpretation
c8yedge_apama_process_cpu_usage	CPU usage by Apama process.	Essential for understanding how Apama impacts overall system CPU resources.
c8yedge_apama_system_cpu_usage	The total CPU usage percentage by the system, including Apama and other process.	Helps gauge the overall CPU load and identify potential bottlenecks.
c8yedge_apama_system_load_average_1m	The system's 1-minute load average, including Apama's impact.	Gives an immediate view of system demand, useful for identifying sudden increases in load.
c8yedge_apama_sag_apama_in_c8y_uptime_secs	This metric measures the uptime of Apama and its correlator in seconds within the Edge deployment.	These are crucial for tracking the stability and reliability of the Apama service.
c8yedge_apama_process_uptime_seconds	Measures how long the Apama process has been running in seconds.	Similar to the uptime metrics above, this provides insights into Apama's process stability.
c8yedge_apama_sag_apama_correlator_uptime_seconds	metrics measure the uptime of Apama and its correlator in seconds within the Edge deployment.	These are crucial for tracking the stability and reliability of the Apama service.
c8yedge_apama_process_files_open_files	The number of files currently open by Apama process.	Critical for ensuring Apama does not run into file descriptor limits, affecting its ability to operate.

Metric	Description	Interpretation
c8yedge_apama_jvm_threads_peak_threads	The peak thread count used by Apama's JVM (Java Virtual Machine).	Indicates the maximum concurrency level required by Apama, useful for JVM tuning and performance optimization.
c8yedge_apama_process_start_time_seconds	The start time of the Apama process, measured in seconds since the Unix epoch.	Can be used to determine the Apama process's age, correlating with other events or metrics.
c8yedge_apama_sag_apama_in_c8y_is_starter_mode	Indicates whether Apama is running in starter mode within Edge.	Starter mode may have different resource usage or limitations compared to full operation mode.
c8yedge_apama_sag_apama_in_c8y_is_safe_mode	Flags if Apama is operating in a safe mode within Edge.	Safe mode might restrict certain operations or functions to ensure stability or security.

MONGODB METRICS

Metrics related to MongoDB, prefixed by **c8yedge_db**

Metric	Description	Interpretation
c8yedge_db_mongodb_up	Indicates whether the MongoDB database instance is up and running.	A binary metric where 1 means the database is operational, and 0 indicates it is down. This metric is crucial for alerting on database availability.
c8yedge_db_process_cpu_seconds_total	Cumulative CPU time used by the database process, measured in seconds.	This helps in understanding the total CPU time the MongoDB process has consumed since it started, allowing for analysis of CPU usage trends over time.

Metric	Description	Interpretation
<code>c8yedge_db_process_resident_memory_bytes</code>	The amount of RAM currently being used by the database process, in bytes.	This metric is vital for understanding the database's memory footprint, helping to ensure that the database does not exceed available memory resources and to plan for scaling or optimization if necessary.
<code>c8yedge_db_process_virtual_memory_bytes</code>	The total virtual memory used by the database process, in bytes.	Virtual memory includes all memory that the process can access, including what is in RAM and on disk (swap). Monitoring this helps in understanding the database's overall memory demand, which is crucial for performance and stability.
<code>c8yedge_db_process_virtual_memory_max_bytes</code>	The maximum amount of virtual memory the database process can use.	This metric often reflects a system or process-level limit on memory usage. While not all systems enforce a maximum virtual memory size, when present, this metric can help identify configurations that may limit database performance or scalability.
<code>c8yedge_db_process_open_fds</code>	The current number of open file descriptors by the database process.	Monitoring this metric alongside <code>c8yedge_db_process_max_fds</code> can warn of potential resource exhaustion if the number of open file descriptors approaches the maximum limit.
<code>c8yedge_db_process_max_fds</code>	The maximum number of file descriptors the database process can open.	File descriptors are used by process to access files and network sockets. This metric indicates the upper limit set for these resources, helping to anticipate and prevent resource exhaustion issues.

Metric	Description	Interpretation
<code>c8yedge_db_process_start_time_seconds</code>	The start time of the database process, measured in seconds since the Unix epoch.	This metric can be used to determine how long the database has been running since its last restart. It's useful for tracking uptime and correlating with other events or metrics.

STATEFULSETS METRICS

This metric tracks the health status of StatefulSets in the Edge deployment. StatefulSets are Kubernetes workloads that manage stateful applications, ensuring that the deployment and scaling of the application are handled properly, and that the application maintains its state across restarts and migrations.

The `c8yedge_statefulset` metric can be qualified by the StatefulSet name, such as `statefulset="c8ycore-sts"`, `statefulset="edge-db-rs0"`, and so on, for monitoring different StatefulSets in the Edge deployment. A gauge value of 1 for a StatefulSet indicates that it is healthy, meaning all desired replicas are up and serving without issues. A value of 0 indicates a failure, such as one or more replicas not running as expected.

Metrics	Label Options
<code>c8yedge_statefulset{statefulset="c8ycore-sts"}</code>	<code>statefulset="c8ycore-sts"</code> <code>statefulset="edge-db-rs0"</code> <code>statefulset="logging-fluentd"</code> <code>statefulset="event-tailer-event-tailer"</code>

DEPLOYMENTS METRICS

These metrics represent the health status of Deployments in Edge. Deployments are another type of Kubernetes workload that manage stateless applications, ensuring that a specified number of replicas of the application are running at any given time.

The `c8yedge_deployment` metric can be qualified by the Deployment name, such as `deployment="c8yedge-operator-controller-manager"`, `deployment="apama-ctrl-scope-edge-deployment"`, and so on, for monitoring different Deployments in Edge. A gauge value of 1 for a Deployment indicates that it is healthy, meaning all desired replicas are up and serving without issues. A value of 0 indicates a failure, such as one or more replicas not running as expected.

Metrics	Label Options
<code>c8yedge_deployment{deployment="microservices-registry"}</code>	<code>deployment="c8yedge-operator-controller-manager"</code> <code>deployment="logging-operator-c8yedge-edge-sample"</code> <code>deployment="psmdb-operator-c8yedge-edge-sample"</code> <code>deployment="microservices-registry"</code> <code>deployment="thin-edge"</code> <code>deployment="smartrule-scope-management-deployment"</code> <code>deployment="apama-ctrl-scope-edge-deployment"</code> <code>deployment="opcua-mgmt-service-scope-management-deployment"</code>

POD METRICS (CPU AND MEMORY USAGE)

These metrics track the CPU and memory usage of the workloads in your Edge deployment. CPU usage is a critical performance metric, indicating how much of the allocated CPU resources are being utilized by a container. Monitoring memory usage helps in managing application scalability, optimizing resource allocation, and preventing out-of-memory (OOM) issues that could lead to application downtime.

The `c8yedge_pod_cpu_usage` and `c8yedge_pod_memory_usage` metrics can be qualified by the Pod and Container names, such as `container="cumulocity-core"`, `pod="c8ycore-sts-0"` and so on for monitoring the CPU and memory usage of different containers in the Edge deployment.

Pod names often contain a unique hash or a set of alphanumeric characters that change every time the pod is restarted, redeployed, or rescheduled on a different node. To continuously monitor the CPU usage of a container across pod restarts, you can use regular expressions (regex) in your monitoring queries to match only the static part of the pod name. For example, if you want to monitor the `apama-ctrl-scope-edge-pod` container regardless of pod restarts, you must construct a query that matches any pod name starting with `apama-ctrl-scope-edge-deployment` without specifying the unique hash, like `container="apama-ctrl-scope-edge-pod",pod=~"apama-ctrl-scope-edge-deployment.*"`.

Metrics

```
c8yedge_pod_cpu_usage{container="cumulocity-core",
pod="c8ycore-sts-0"}
```

Label Options

- container="cumulocity-core", pod="c8ycore-sts-0"
- container="openresty", pod="c8ycore-sts-0"
- container="smartrule-scope-management-pod", pod=~"smartrule-scope-management-deployment-"
- container="apama-ctrl-scope-edge-pod", pod=~"apama-ctrl-scope-edge-deployment-"
- container="opcua-mgmt-service-scope-management-pod", pod=~"opcua-mgmt-service-scope-management-deployment-"
- container="mongod", pod="edge-db-rs0-0"
- container="mongodb-exporter", pod="edge-db-rs0-0"
- container="psmdb-operator", pod=~"psmdb-operator-c8yedge-edge-sample-"
- container="kedge-agent", pod=~"thin-edge-"
- container="mosquitto", pod=~"thin-edge-"
- container="microservices-registry", pod=~"microservices-registry-"
- container="microservices-registry", pod=~"microservices-registry-garbage-collector-"
- container="microservices-registry-config", pod=~"microservices-registry-config-"
- container="fluentd", pod="logging-fluentd-0"
- container="config-reloader", pod="logging-fluentd-0"
- container="fluent-bit", pod=~"logging-fluentbit-"
- container="event-tailer", pod="event-tailer-event-tailer-0"
- container="logging-operator", pod="logging-operator-c8yedge-edge-sample-"
- container="manager", pod=~"c8yedge-operator-controller-manager-"

```
c8yedge_pod_memory_usage{container="cumulocity-
core", pod="c8ycore-sts-0"}
```

- container="cumulocity-core", pod="c8ycore-sts-0"
- container="openresty", pod="c8ycore-sts-0"
- container="smartrule-scope-management-pod", pod=~"smartrule-scope-management-deployment-"
- container="apama-ctrl-scope-edge-pod", pod=~"apama-ctrl-scope-edge-deployment-"
- container="opcua-mgmt-service-scope-management-pod", pod=~"opcua-mgmt-service-scope-management-deployment-"
- container="mongod", pod="edge-db-rs0-0"
- container="mongodb-exporter", pod="edge-db-rs0-0"
- container="psmdb-operator", pod=~"psmdb-operator-c8yedge-edge-sample-"
- container="kedge-agent", pod=~"thin-edge-"
- container="mosquitto", pod=~"thin-edge-"
- container="microservices-registry", pod=~"microservices-registry-"
- container="microservices-registry", pod=~"microservices-registry-garbage-collector-"
- container="microservices-registry-config", pod=~"microservices-registry-config-"
- container="fluentd", pod="logging-fluentd-0"
- container="config-reloader", pod="logging-fluentd-0"
- container="fluent-bit", pod=~"logging-fluentbit-"
- container="event-tailer", pod="event-tailer-event-tailer-0"
- container="logging-operator", pod="logging-operator-c8yedge-edge-sample-"
- container="manager", pod=~"c8yedge-operator-controller-manager-"

INSTALLING AND CONFIGURING MONITORING TOOLS**INSTALLING PROMETHEUS**

Prometheus is an open-source project that is used for monitoring application state. See <https://prometheus.io/> for detailed information on Prometheus and how to use it. See [Installing Prometheus](#) for detailed steps on installing Prometheus.

INSTALLING GRAFANA

Grafana is an open-source project which serves as an introductory tool for querying, visualising, alerting, and exploring metrics, logs, and traces from diverse storage locations. See <https://grafana.com/docs/grafana/latest/> for detailed information on Grafana and how to use it. See [Installing Grafana](#) for detailed steps on installing Grafana.

MONITORING THE EDGE METRICS FROM YOUR CLOUD TENANT

In your Cumulocity cloud tenant, you can monitor the measurements of the Edge listed in the table below. To monitor the measurements from your cloud tenant, ensure that you have registered your Edge with the Cumulocity cloud tenant. See [Registering Edge in the cloud tenant](#) for more information.

Measurement	Metrics	Description
Disk space	- Total disk space - Free disk space - Used disk space - Percentage of used disk space	The Edge appliance sends the disk space metrics as a measurement for both installation disk and data disk, every 10 minutes. The measurements are sent in gigabytes (GB) rounded to two decimal places. The percentage is rounded to one decimal place. The data points for this measurement are: - <i>c8y_InstallationDisk</i> - <i>c8y_DataDisk</i> If Edge is unable to read the metrics from the installation disk or the data disk, an alarm is sent to the Cumulocity tenant. The alarms have a minor severity and the data points for the alarms are: - <i>c8y_FileSystemMeasurementErrorInstallationDisk</i> - <i>c8y_FileSystemMeasurementErrorDataDisk</i>
Memory (RAM)	- Total RAM - Free RAM - Used RAM - Percentage of RAM used	The Edge appliance sends the memory usage metrics as a measurement every 5 seconds in gibibytes (GiB). The data point for this measurement is <i>c8y_Memory</i> If Edge is unable to read the metrics from the memory, an alarm is sent to the Cumulocity tenant. The data point for the alarm is: - <i>c8y_MemoryMeasurementError</i>.
CPU	Percentage of CPU used Unit: Percentage	The Edge appliance sends the percentage of CPU used at intervals over 5 seconds, 60 seconds, and 600 seconds. The data points for this measurement are: - <i>c8y_CpuUsage5Seconds</i> - <i>c8y_CpuUsage60Seconds</i> - <i>c8y_CpuUsage600Seconds</i> If Edge is unable to read the metrics from the CPU, an alarm is sent to the Cumulocity tenant. The data point for the alarm is: - <i>c8y_CPUMeasurementError</i>.
Disk I/O	- Data read per second - Data written per second Unit: KB/s	The Edge appliance sends the disk input/output metrics as a measurement for both installation disk and data disk at intervals over 5 seconds, 60 seconds, and 600 seconds. The data points for this measurement are: - <i>c8y_DataDiskIo5Seconds</i> - <i>c8y_DataDiskIo60Seconds</i> - <i>c8y_DataDiskIo600Seconds</i> - <i>c8y_InstallationDiskIo5Seconds</i> - <i>c8y_InstallationDiskIo60Seconds</i> - <i>c8y_InstallationDiskIo600Seconds</i> If Edge is unable to read the metrics from the disk, an alarm is sent to the Cumulocity tenant. The data point for the alarm is: - <i>c8y_DiskIoMeasurementError</i>.
Network	- Data and packets sent per second - Data and packets received per second Unit: KB/s and packets/s	The Edge appliance sends the network metrics as a measurement at intervals over 5 seconds, 60 seconds, and 600 seconds. The data points for this measurement are: - <i>c8y_NetworkInterface_Io-5Seconds</i> - <i>c8y_NetworkInterface_Io-60Seconds</i> - <i>c8y_NetworkInterface_Io-600Seconds</i> If Edge is unable to read the metrics from the network, an alarm is sent to the Cumulocity tenant. The data point for the alarm is: - <i>c8y_NetworkIoMeasurementError</i>.

To monitor the metrics in your Cumulocity tenant, you can create a dashboard and add widgets in the Cockpit application of your tenant. For more

information about creating dashboards, see [Working with dashboards](#).

Also, you can define smart rules to create alerts or raise alarms for the metrics. For example, when the free disk space is less than 5 GB, create an alert. For more information about smart rules, see [Smart rules](#).

TROUBLESHOOTING

EDGE CR STATUS VALIDATIONLOOPBACKOFF

After deploying or updating Edge, if the Edge CR status shows as "ValidationLoopBackOff" as shown below:

Output of `kubect1 get edge c8yedge -n c8yedge` :

NAME	DOMAIN NAME	VERSION	STATUS
c8yedge	myown.iot.com	1018.0.1	ValidationLoopBackOff

Describe the Edge CR (c8yedge): `kubect1 describe edge c8yedge -n c8yedge` .

In the events section, you will find the reasons for the validation failure. For example:

```

Name: c8yedge
Namespace: c8yedge
API Version: edge.cumulocity.com/v1
Kind: CumulocityIoTEdge

Status:
  Deploying Generation: x
  State: ValidationLoopBackOff

Events:
  Type    Reason          Age    From          Message
  ----    -
  Normal  Validating      8m37s  cumulocityiotedge validating
  Normal  ValidationLoopBackoff  8m37s  cumulocityiotedge invalid [spec.licenseKey] value [empty]

```

EDGE CR STATUS INSTALLLOOPBACKOFF OR UPDATELOOPBACKOFF

After deploying or updating Edge, if the Edge CR status is `InstallationLoopBackOff` or `UpdateLoopBackOff` , as part of the output of `kubect1 get edge c8yedge -n c8yedge` :

NAME	DOMAIN NAME	VERSION	STATUS
c8yedge	myown.iot.com	1018.0.1	InstallLoopBackOff

Describe the Edge CR (cumulocity-iot-edge) with the command `kubect1 describe edge c8yedge -n c8yedge` .

In the events section, you will find the reasons for the install or update failure. For example:

Name: c8yedge
Namespace: c8yedge
API Version: edge.cumulocity.com/v1
Kind: CumulocityIoTEdge

Status:

Deploying Generation: x
State: InstallLoopBackOff

Events:

Type	Reason	Age	From	Message
Normal	Validating	8m37s	cumulocitiotedge	validating
Normal	ValidationPassed	8m37s	cumulocitiotedge	validation passed
Normal	Installing	8m34s	cumulocitiotedge	installing
Normal	InstallLoopBackOff	1m00s	cumulocitiotedge	waiting for mongo server to be ready

Check the logs of the Edge operator, MongoDB and core pods to get more insights.

The Edge operator logs:

```
kubectll logs -n c8yedge deployment.apps/c8yedge-operator-controller-manager -c manager
```

MongoDB logs:

```
kubectll logs -n c8yedge --all-containers --prefix statefulset.apps/edge-db-rs0
```

Core logs:

```
kubectll logs -n c8yedge --all-containers --prefix statefulset.apps/c8ycore-sts
```

PENDING EXTERNAL CUMULOCITY-CORE SERVICE IP

The external IP is not assigned to the `cumulocity-core` service.

Output of `kubectll get service cumulocity-core -n c8yedge`:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
cumulocity-core	LoadBalancer	X.X.X.X **REDACTED	<pending>	443:31342/TCP,1883:32751/TCP,8883:32270/TCP	12m

For more information, see [Assigning an external IP](#).

EDGE NOT SHOWING UP AS A DEVICE IN THE CLOUD TENANT

Edge is unable to register itself as a device in your cloud tenant.

Describe the Edge CR `kubectll describe edge c8yedge -n c8yedge`

In the events section, you will find an event with a reason. For example:

Name: c8yedge
Namespace: c8yedge
API Version: edge.cumulocity.com/v1
Kind: CumulocityIoTEdge

Status:
Deployed Generation: x
State: Ready

Events:

Type	Reason	Age	From	Message
Normal	Validating	8m37s	cumulocityiotedge	validating
Normal	ValidationPassed	8m37s	cumulocityiotedge	validation passed
Normal	Installing	8m34s	cumulocityiotedge	installing
.....				
Normal	Ready	1m00s	cumulocityiotedge	installed successfully
Warning	FailedToRegisterCloudDevice	0m30s	cumulocityiotedge	Device failed to connect to <cloud tenant domain>, you might not have uploaded the device certificate to Cumulocity

Possible reasons:

- You didn't add the Certificate Authority (CA) certificate to the trusted certificate list of your tenant. For completing the registration process, sign into your cloud tenant and follow the steps described in [Managing trusted certificates](#) to add the Certificate Authority (CA) certificate to the trusted certificate list of your tenant. For more details, see [Device certificates](#). For more information see [Connecting Edge to the cloud](#).
- The specified cloud tenant domain is not reachable or incorrect.

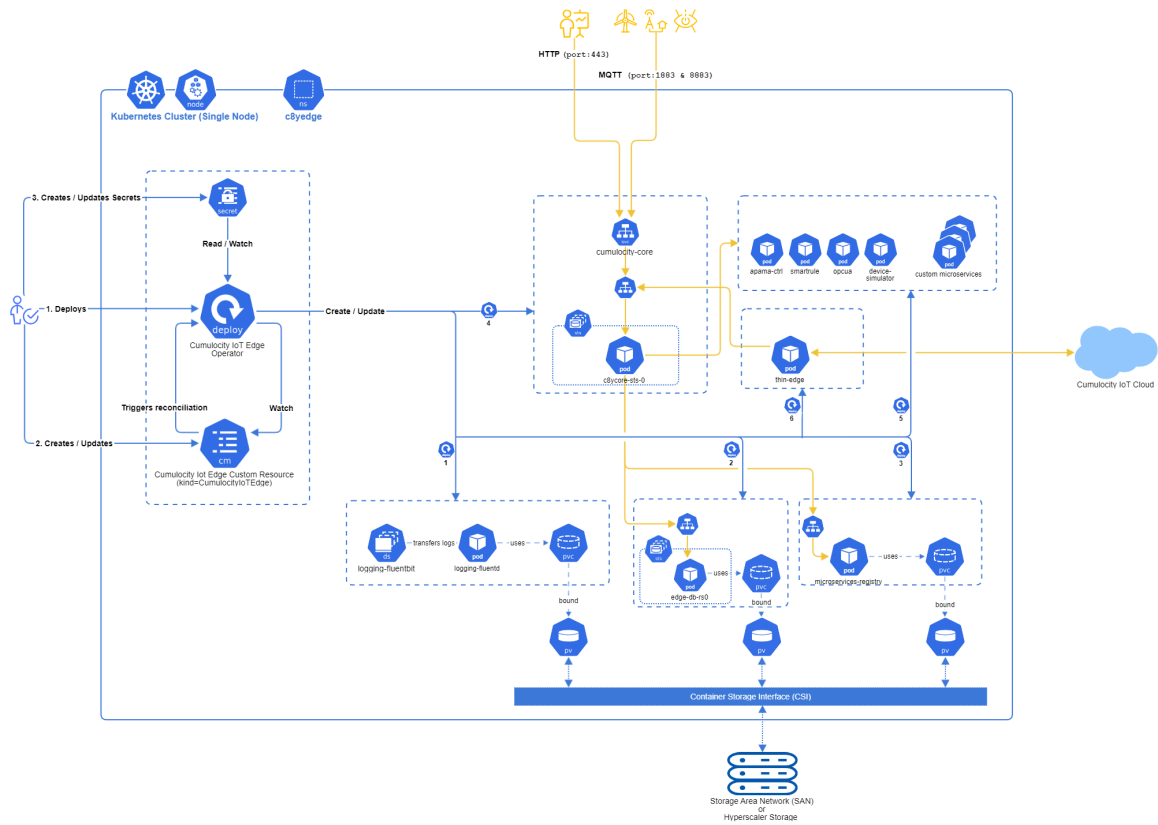
If you must contact [product support](#), include the output of the diagnostics dump. For more information about accessing diagnostic logs, see [Accessing logs](#).

DEPLOYMENT ARCHITECTURE

The Edge operator operates within a namespace (default **c8yedge**, unless specified during the Edge operator installation). It creates and monitors multiple Kubernetes resources that collectively constitute the Edge.

You can deploy Edge on Kubernetes in two modes: one includes MongoDB, which is both deployed and managed by the Edge operator; the other involves connecting to an externally deployed MongoDB.

The deployment model is depicted below:



CONFIGURING EDGE

Learn how to configure many different aspects of Edge.

CONFIGURING REMOTE CONNECTIVITY

For information about connecting your Edge appliance to the cloud tenant, see [Connecting Edge to the cloud](#).

CONFIGURING THE EMAIL SERVER

Configuring an email server enables you to receive email notifications about events, alarms, and also to reset your password. In case you forget the password, the Edge appliance mails you the password reset link to reset your password.

To configure the "reset password" template and email server settings, perform the following steps:

1. Log into the Management tenant using the management domain (for example, `management.myown.iot.com`) and administrator credentials created during the installation. For more information on logging in to the Management tenant, see [To access Edge](#)
2. Update the email server details and templates as mentioned in [Password reset](#) and [Email server](#).

CONNECTING DEVICES TO EDGE

INTRODUCTION

Edge uses REST and MQTT as protocols for external communication. Both protocols may be used to interface devices with Edge. For more information, see [Device integration using REST](#) and [SmartREST 2.0](#).

Additionally, Edge offers:

- OPC UA protocol. OPC UA protocols support through the OPC UA device gateway and OPC UA management service.
- Cloud Fieldbus functionality to collect data from fieldbus devices and remotely manage them. For example, Modbus protocol. With the release of Cumulocity Edge version 10.18, we are announcing the deprecation of the Cumulocity Linux Agent included in the Cumulocity Edge offering. For further details, see the announcement in the [10.18 release notes](#).

INFO

Currently, only OPC UA protocols are supported.

USING EDGE

GETTING STARTED

Read about the concepts behind the Cumulocity platform. Get familiar with the platform UI or directly get started by connecting your phone.

See [Getting started](#).

BRANDING

With the Branding feature, you can fully customize the look of your tenant to your own preferences. To access the Branding feature:

Log into the Management tenant using the management domain (for example, `management.myown.iot.com`) and administrator credentials created during the installation. For more information on logging in to the Management tenant, see [To access Edge](#)

Switch to the Administration application using the application switcher at the right of the top bar .

Go to **Settings > Enterprise tenant** and follow the instructions detailed in [Branding](#).

DEVELOPING CUMULOCITY WEB APPLICATIONS

If you develop a Cumulocity web application using the [Web SDK](#), for an Edge appliance configured with a certificate not trusted by Node.js (for example, a self-signed certificate), then you must ensure that Node.js trusts the root certificate.

To add the Edge appliance's self-signed certificate to the Node.js trust store, set the environment variable `NODE_EXTRA_CA_CERTS` to the path of the certificate before executing the `npm` commands.

Example:

On Windows Powershell:

```
$env:NODE_EXTRA_CA_CERTS=<path-to-the-certificate>\certificate.pem
```

On Windows command prompt:

```
set NODE_EXTRA_CA_CERTS=<path-to-the-certificate>\certificate.pem
```

On Linux platforms:

```
export NODE_EXTRA_CA_CERTS=<path-to-the-certificate>/certificate.pem
```

For more information on the Cumulocity web application development, see [Web SDK](#).

WORKING WITH DATAHUB

ANALYTICAL QUERYING WITH DATAHUB EDGE

DataHub Edge is an optional component of Edge. DataHub Edge complements the ad-hoc querying of recent device data with analytical querying over long periods of time. For that purpose, data is moved from the Operational Store of Edge to a local data lake, with the data being stored in a concise and query-efficient format. DataHub Edge allows you to run SQL queries against the data lake contents so that you can gain more insights into your device data.

DataHub Edge is the counterpart of DataHub, the variant for cloud deployments. To learn more about DataHub in general, see [DataHub overview](#). To learn more about DataHub Edge, see [Running DataHub on the Edge](#).

 > Edge on Kubernetes > Working with DataHub

EDGE ON KUBERNETES

This document describes the conceptual aspects of [Cumulocity Edge](#) on Kubernetes, the single-server variant of the Cumulocity platform. For more information about Edge, see [Cumulocity Edge](#).

OVERVIEW

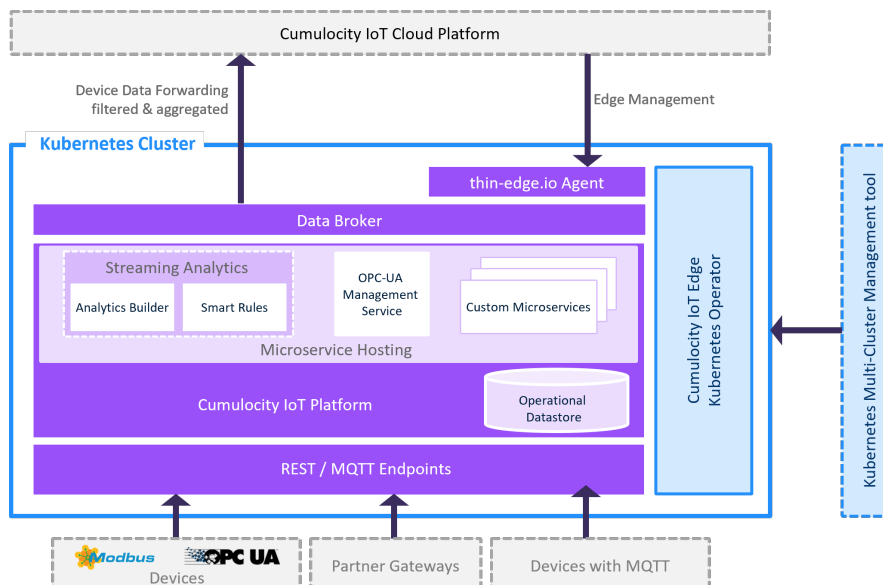
Edge on Kubernetes represents a cloud-native solution for the delivery, deployment, and management of Edge. It retains the functionalities of the Edge appliance VM, encompassing similar benefit like autonomy, data reduction, and reactivity.

Kubernetes offers an efficient platform for deploying, scaling, and managing containerized applications using a centralized control plane. Given this, containers orchestrated by Kubernetes have become standard in contemporary IT, fitting a broad spectrum of deployment contexts. Edge on Kubernetes capitalizes on these orchestration and management benefits, streamlining the operations of the Edge servers with other containerized applications in your deployment landscape.

Edge on Kubernetes captures all the highlighted advantages of a cloud-native deployment strategy by using a Kubernetes Operator, known as the Edge operator. This Edge operator serves as the central controller and facilitates the deployment and management of the Edge on a single-node Kubernetes cluster. This equips you with:

- **Unified Management:** Manage Edge through an Edge Custom Resource (CR) in accordance with standard Kubernetes resources. This approach provides a unified and familiar interface for administrators and operators.
- **Desired State Management:** Specify the desired state of Edge through Edge CR. The Edge operator then ensures that the actual state matches the desired state, simplifying operations and reducing the need for manual intervention.
- **GitOps Friendly:** The declarative nature of CR aligns seamlessly with GitOps workflows. The entire Edge deployment state, including configurations, can be stored and versioned in Git, promoting transparency and traceability.
- **Automation:** Edge CR empowers you to automate various operational aspects of Edge like scaling, upgrades, and configuration.
- **Distribution:** Distribute and replicate the Edge deployments using Edge CR. Simplify the deployment and accelerate the adoption of Cumulocity platform across diverse environments.

Although the features remain on par with the Edge appliance VM, the deployment strategy undergoes a change, as illustrated in the diagram below:



KUBERNETES OPERATOR

Kubernetes Operators are a powerful concept and pattern for managing complex applications and services within Kubernetes clusters. The operators are software extensions to Kubernetes that make use of [custom resources](#) to manage applications and their components. The operators follow Kubernetes principles, notably the [control loop](#).

The integral part of the operator pattern is Custom Resource Definitions (CRDs), which allow you to define application-specific resources and its behavior. An operator uses these CRDs to understand how to manage and interact with the application.

The operators are particularly valuable for managing stateful applications, like Cumulocity. The operators automate best practices, routine tasks, and provide a consistent way to manage complex applications across different environments.

EDGE OPERATOR

The Edge operator automates the deployment and management of Edge on Kubernetes. The Edge operator manages a Custom Resource Definition (CRD) to extend the Kubernetes API for Edge. You can deploy and manage Edge on a Kubernetes cluster through Edge Custom Resource (CR). Modify the Edge CR file and use `kubectl apply -f` to apply the changes. The Edge operator receives these changes and installs or updates Edge accordingly.

The Edge operator enables you to install, configure, upgrade or downgrade, scale up or down Edge on Kubernetes. The Edge operator supports:

- Deploy Edge
- Version upgrade and downgrade
- Scale up and down resources
- Configure persistent storage
- Validate configurations

VERSIONS

Cumulocity Edge 10.18 uses the following versions:

Edge component	Version
Cumulocity platform	10.18.0
Streaming Analytics	10.18